

Dense SLAM Using 3-D Gaussian Splatting With IMU and Geometric Constraints

Ruijie Che, Hao Hu, Ningbo Wang[✉], and Zishen Li[✉]

Abstract—Simultaneous localization and mapping (SLAM) is a foundational technology for autonomous systems, augmented reality, and robotics, enabling real-time localization and environment perception. Recently, Gaussian splatting has been widely adopted in SLAM frameworks due to its capabilities for explicit geometric representation, real-time photorealistic rendering, and efficient differentiable optimization. However, although existing monocular methods exploit line features and inertial measurement units (IMUs) to improve Gaussian reconstruction, errors introduced during IMU initialization and the lack of depth supervision during optimization often lead to uncertain artifacts, limiting overall system performance. To address these limitations, we propose a tightly-coupled monocular-inertial framework that leverages high-frequency IMU observations to enhance both localization and Gaussian mapping. Our approach incorporates an efficient IMU initialization scheme and utilizes both point-line features and IMU observations for robust tracking. Moreover, IMU-supervised depth estimation and normal maps provide metric-scale geometric constraints, which are further combined with structural lines to generate planar hypotheses, densifying Gaussian primitives in low-texture regions. A pyramid-based Gaussian optimization framework is then employed, where multidimensional losses drive the progressive refinement of both global geometric structures and local texture details. Extensive evaluations on the Replica, TUM RGB-D, ScanNet, OpenLORIS, and KITTI datasets demonstrate that our method achieves more stable tracking and higher-quality reconstructions than state-of-the-art approaches, while maintaining real-time performance.

Index Terms—Depth estimation, Gaussian splatting, geometric constraints, visual-inertial SLAM.

Received 16 January 2026; accepted 4 February 2026. Date of publication 9 February 2026; date of current version 27 April 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2024YFB3909100, in part by the National Natural Science Foundation of China under Grant 42404013, in part by Beijing Nova Program under Grant 20230484329, in part by Jinan Haiyou Leading Talents of Industry, in part by the Youth Innovation Promotion Association and Future Star Program of Chinese Academy of Sciences, and in part by Shandong Key Research and Development Program under Grant 2024CXGC010116. (Corresponding author: Zishen Li.)

Ruijie Che is with the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100094, China, also with the School of Electronic, Electrical and Communication Engineering, University of Chinese Academy of Sciences, Beijing 101408, China, and also with the Qilu Aerospace Information Research Institute, Jinan 250100, China (e-mail: cheruijie22@mails.ucas.ac.cn).

Hao Hu is with Shandong Normal University, Jinan 250358, China, and also with the Qilu Aerospace Information Research Institute, Jinan 250132, China (e-mail: huhao199709@163.com).

Ningbo Wang is with the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100094, China (e-mail: wangnb@aircas.ac.cn).

Zishen Li is with the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100094, China, and also with the Qilu Aerospace Information Research Institute, Jinan 250132, China (e-mail: lizishen@aircas.ac.cn).

Digital Object Identifier 10.1109/IJOT.2026.3662499

I. INTRODUCTION

SIMULTANEOUS localization and mapping (SLAM) is a foundational technology in computer vision and robotics, enabling autonomous navigation and environmental understanding for robots. Traditional SLAM methods [1], [2], [3], [4], [5] primarily focus on real-time tracking and geometric mapping through multisensor fusion. Although these methods achieve accurate performance through bundle adjustment or filtering algorithms, their underlying 3-D representations (e.g., point clouds, meshes, and surfels) exhibit limitations in density, photorealistic rendering capabilities, and geometric completeness, which constrain applications such as immersive AR/VR or high-fidelity scene reconstruction.

Recently, 3-D Gaussian splatting [6] has emerged as a breakthrough technique for photorealistic 3-D scene reconstruction. Its explicit representation through anisotropic 3-D Gaussians coupled with tile-based rasterization enables both highly efficient differentiable optimization and real-time photorealistic rendering, which motivates the integration of 3DGS into SLAM pipelines [7], [8], [9]. However, current Gaussian SLAM approaches are still predominantly vision-only. Monocular systems suffer from scale ambiguity, stereo/RGB-D setups are constrained by limited sensing ranges, and purely visual pipelines often fail in man-made, low-texture scenes due to feature scarcity, leading to unstable initialization, tracking drift, and degraded reconstructions. Existing extensions such as MG-SLAM [10] attempt to exploit line features, but they neither incorporate IMU constraints nor fully utilize line features for mapping, and depend on restrictive assumptions (Manhattan world), limiting robustness in diverse environments.

To address these limitations, we propose a monocular-inertial Gaussian-SLAM framework that tightly integrates point-line features with IMU constraints for both tracking and mapping. IMU measurements provide metric-scale recovery and high-frequency motion priors, while structural line features offer reliable constraints in man-made, low-texture environments where point features alone are insufficient. By jointly exploiting point, line, and IMU information, our system achieves robust and accurate tracking even under challenging conditions. In addition to stable tracking, IMU initialization is also crucial. It ensures fast readiness for downstream applications like augmented reality while accelerating convergence in bundle adjustment and Gaussian splatting optimization [11], [12]. To this end, we propose an efficient vision-inertial initialization algorithm. First, we convert corresponding point

and line features between consecutive frames into rotational constraints and optimize the gyroscope bias by incorporating gyroscope observations. Then, the accelerometer bias and gravity are recovered via an analytical solution, which is faster than iterative approaches. Finally, all inertial parameters are refined through maximum a posteriori (MAP) estimation in the back end.

In Gaussian SLAM, the initialization of Gaussian primitives is typically performed using sparse points derived from feature point triangulation [7], which is often unreliable, especially under monocular settings. To mitigate this, pretrained 3-D models based on limited 3-D samples offer a viable initialization strategy [13]. Nevertheless, current unsupervised depth estimation models suffer from accuracy limitations and fundamentally lack the capability to predict accurate metric depth [14], [15], [16], [17], [18]. Inspired by [19], we adopt an IMU-supervised monocular depth estimation model to provide metric-scale depth, which serves as an enabling component for our geometry-aware Gaussian mapping framework. This pretrained network exhibits generalization capability, allowing it to be directly applied to diverse SLAM scenarios without fine-tuning. To further improve mapping in low-textured structured environments, we integrate line features with proxy depth and normal maps to generate consistent 3-D plane hypotheses. By evaluating the spatial feature distribution on reconstructed planes, we identify under-constrained regions and systematically densify them by inserting supplementary feature points both in planar interiors and along structural edges defined by the plane hypotheses. These augmented features provide denser and more reliable constraints for Gaussian initialization and optimization, significantly enhancing planar surface completeness, geometric accuracy, and view-synthesis quality.

Finally, we adopt a pyramid-based Gaussian optimization strategy that progressively optimizes Gaussian primitives with multiscale regularization, including color, depth, normal, and geometry-based constraints (line and plane). This multilevel optimization balances efficiency with accuracy, ensuring both global consistency and fine-grained detail in the reconstructed scene.

In summary, our contributions are as follows.

- 1) We propose a point–line–IMU collaborative initialization and tracking strategy. By jointly leveraging point and line epipolar constraints with IMU preintegration, our method rapidly calibrates inertial biases, recovers metric scale, and initializes the global coordinate system. A tightly-coupled factor-graph optimization is further employed to minimize residuals and ensure robust and accurate tracking.
- 2) We introduce a geometry-aware Gaussian mapping framework that combines IMU-supervised metric-scale depth estimation and structural plane hypotheses for geometry-constrained Gaussian initialization, together with a pyramid-based Gaussian optimization strategy that incorporates multiscale refinement and multidimensional regularization with enhanced line constraints. This approach improves structural fidelity and photorealistic quality in the reconstructed scenes.

- 3) We build a monocular–inertial Gaussian-SLAM system that fuses point–line features and IMU measurements for both tracking and mapping. Extensive experiments on Replica, ScanNet, TUM RGB-D, and OpenLORIS demonstrate competitive performance in tracking and photorealistic reconstruction.

II. RELATED WORK

A. Traditional SLAM

SLAM has long been a central research problem in robotics and computer vision. Early classical methods [4], [5], [20], [21] primarily relied on sparse point features to achieve accurate camera tracking and geometric mapping. To resolve the inherent scale ambiguity of cameras and to improve robustness under challenging conditions, visual–inertial frameworks [22], [23] were later introduced. By tightly coupling point features with high-frequency inertial measurements, these systems significantly enhance localization stability and enable metric-scale estimation. Nevertheless, their strong dependence on point features still causes degraded performance in low-texture or repetitive-pattern scenarios. To address these limitations, researchers have explored richer geometric primitives. Line features provide additional constraints that help maintain tracking continuity in weakly textured regions, while planar constraints introduce higher-level structural consistency into the optimization process. By combining points, lines, and planes, hybrid frameworks [24], [25], [26], [27] are able to achieve more reliable pose estimation and more complete scene reconstruction, especially in man-made environments. In parallel, direct and semi-direct methods [1], [2], [3] emerged as an alternative to explicit feature extraction. These approaches optimize photometric consistency directly at the pixel level, offering denser constraints that are particularly beneficial when traditional feature detection fails. However, they are inherently more sensitive to illumination changes and typically require higher computational resources, which limits their deployment in real-time, resource-constrained platforms. Overall, traditional SLAM systems primarily focus on tracking and geometric reconstruction, while our approach also considers high-fidelity appearance reconstruction.

B. Neural Implicit Based SLAM

Following the rise of NeRF [28], neural implicit scene representations have been actively explored within the SLAM domain. By modeling scenes as continuous volumetric functions, these methods enable photorealistic novel view-synthesis and high-fidelity reconstruction, which are difficult to achieve with traditional pipelines. iMAP [29] first demonstrated the feasibility of implicit representations for SLAM by employing a single multilayer perceptron (MLP) to jointly optimize scene geometry and camera poses in real time. NICE-SLAM [30] improved scalability by introducing a hierarchical grid representation, allowing for longer trajectories in large-scale environments. Vox-Fusion [31] further accelerated optimization through efficient voxel-based encodings. GO-SLAM [32] improved robustness in challenging environments, Co-SLAM [33] enhanced scalability through collaborative

optimization, and ESLAM [34] advanced real-time performance with efficient implicit representations. Beyond these foundational systems, a variety of subsequent works have further extended neural implicit SLAM, exploring broader application scenarios such as large-scale environments, multiagent settings, and enhanced robustness under real-world conditions [35], [36], [37], [38]. However, NeRF-based SLAM systems remain limited by several factors. The high computational cost of volume rendering prevents real-time deployment on resource-constrained platforms. The lack of explicit geometry makes it difficult to incorporate structural cues such as points, lines, and planes, or to effectively integrate inertial measurements. Moreover, implicit representations often struggle with incremental updates, loop closure, and long-term consistency, which are essential for robust SLAM in practical scenarios.

C. Gaussian Splatting-Based SLAM

The recently proposed 3-D Gaussian splatting [6] represents scenes using anisotropic Gaussian primitives with learnable positions, scales, and colors, enabling novel view-synthesis and 3-D reconstruction. Unlike NeRF, this explicit formulation supports real-time differentiable rendering while retaining geometric interpretability, making it particularly attractive for SLAM applications. MonoGS [8] first demonstrated the feasibility of applying 3-D Gaussian splatting to monocular reconstruction. SplatAM [9] extended this idea by employing Gaussians with spatial boundary control for dynamic map expansion. Photo-SLAM [7] integrated a sparse-feature-based front end with photorealistic mapping and introduced a pyramid optimization strategy. Gaussian-SLAM [39] further extended 3DGS to photorealistic dense SLAM, enabling dynamic mapping and robust frame-to-model tracking through photometric error minimization. MG-SLAM [10] introduced structural assumptions to improve system performance in indoor environments. In addition, a variety of extensions [40], [41], [42], [43], [44], [45] have been proposed to broaden the applicability of Gaussian splatting-based SLAM, addressing challenges such as semantics, adverse weather, dynamic scenes, and low-light conditions.

However, despite these advances, existing Gaussian splatting-based SLAM systems still struggle in low-texture man-made environments. In such scenarios, weak textures often lead to unstable front-end tracking and incomplete back-end reconstruction, especially when large planar regions dominate the scene. Although inertial measurements can mitigate some issues, current methods still suffer from unstable IMU initialization and provide little support for mapping. To address these challenges, we propose a monocular-inertial Gaussian-SLAM framework. In the front end, our system employs an efficient initialization scheme and tightly-coupled tracking based on points, lines, and inertial measurements within a factor-graph optimization. In the back end, IMU-supervised depth estimation and normal maps provide metric-scale constraints to guide the densification of Gaussian primitives and enhance geometric reconstruction. Together, these designs significantly improve the robustness, efficiency, and reconstruction fidelity of Gaussian splatting-based SLAM in low-texture indoor environments.

III. METHODS

The overall architecture of the proposed system is illustrated in Fig. 1. Compared with existing methods [10], our approach fully exploits multimodal fusion and IMU-based cross-sensor constraints, while extending the applicability of line features, thereby enabling faster and more robust initialization in texture-poor environments. In addition, we propose a plane detection algorithm that does not rely on the Manhattan assumption, together with a Gaussian element-based density optimization strategy, which significantly broadens the applicability of the system to diverse scenarios.

In particular, in Section III-B, we introduce a point-line-IMU collaborative initialization and tracking strategy, which enables rapid and accurate system initialization and achieves robust pose estimation through tightly-coupled factor-graph optimization. In Section III-C, we present a geometry-aware density optimization framework that leverages IMU constraints for metric-scaled depth estimation, planar hypothesis construction, and geometry-driven densification, thereby providing precise and dense point clouds for Gaussian initialization. In Section III-D, we combine pyramid-based progressive optimization with a multidimensional loss function, which not only ensures global consistency but also strengthens structural constraints on edges, ultimately yielding highly accurate and photorealistic reconstruction results.

A. Preliminaries

1) *IMU Preintegration*: An inertial measurement unit (IMU) is a sensor that measures motion-related quantities, typically providing measurements of linear acceleration $\tilde{a}$ and angular velocity $\tilde{\omega}$ using accelerometers and gyroscopes. IMU measurements offer complementary motion information to visual observations and are widely used to improve robustness in motion state estimation. We denote the true kinematic states as a and ω , respectively. The superscripts b and w indicate that the vectors are expressed in the body (IMU) frame or the world frame. Considering the presence of sensor errors, the relationship between the measurements and the true states can be expressed as

$$\begin{aligned}\tilde{a}^b &= R_w^b(a^w + g^w) + b_a + n_a \\ \tilde{\omega}^b &= \omega^b + b_g + n_g\end{aligned}\quad (1)$$

where b_g and b_a denote the gyroscope and accelerometer biases, respectively. n_g and n_a represent zero-mean Gaussian noise, and g^w is the gravity vector in the world frame. R_w^b is the rotation matrix from the world frame to the body frame.

Let $\{p_{b_i}^w, q_{b_i}^w\}$, and v_t^w denote the translation, rotation, and velocity of the IMU body at time t in the world frame, where $q_{b_i}^w$ is represented as a quaternion. The time derivatives of the state variables can be formulated as

$$\begin{aligned}\dot{p}_{b_i}^w &= v_t^w \\ \dot{v}_t^w &= a_t^w \\ \dot{q}_{b_i}^w &= \frac{1}{2}q_{b_i}^w \otimes \begin{bmatrix} \omega_{b_i} \\ 0 \end{bmatrix}\end{aligned}\quad (2)$$

where $\otimes$ denotes quaternion multiplication.

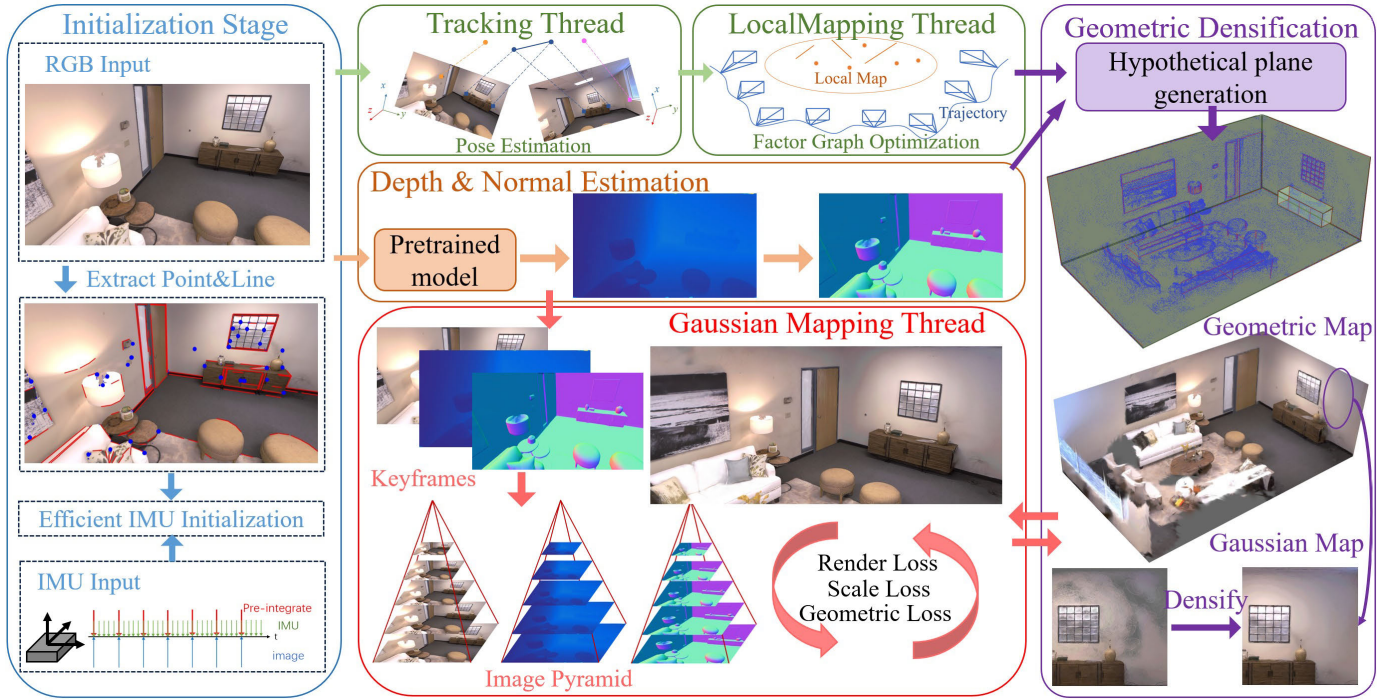


Fig. 1. Overview of the proposed framework. The system begins with multisensor initialization, where IMU measurements are preintegrated and combined with point and line features extracted from RGB images to achieve efficient IMU-aided initialization. Camera poses are then estimated and refined through local optimization, ensuring consistent trajectories and local maps. Meanwhile, a pretrained network predicts depth maps for incoming keyframes, from which normal maps are subsequently derived. These depth and normal maps are leveraged by a geometric densification process to generate hypothetical planes that capture structural regularities and fill in missing regions, thereby enhancing geometric completeness. All these outputs are then incorporated into Gaussian mapping, where a pyramid-based hierarchical optimization is applied to adjust 3-D Gaussians under rendering, scale, and geometric constraints. This sequential process enables accurate pose estimation and high-quality reconstruction in real time.

By integrating IMU measurements from time i to j , the continuous IMU motion dynamics can be expressed as

$$\begin{aligned} p_{b_j}^w &= p_{b_i}^w + v_i^w \Delta t + \iint_{t \in [i,j]} [R_{b_i}^w (\tilde{a}^{b_i} - b_a) - g^w] dt^2 \\ v_j^w &= v_i^w + \int_{t \in [i,j]} [R_{b_i}^w (\tilde{a}^{b_i} - b_a) - g^w] dt \\ q_{b_j}^w &= q_{b_i}^w \otimes \int_{t \in [i,j]} \frac{1}{2} \Omega(\tilde{\omega}^{b_i} - b_\omega) q_t^{b_i} dt \end{aligned} \quad (3)$$

where

$$\Omega(\omega) = \begin{bmatrix} -\omega^\wedge & \omega \\ -\omega^T & 0 \end{bmatrix},$$

$\omega^\wedge$ denotes the skew-symmetric matrix of ω , and Δt is the time interval between i and j .

Since IMU data are collected at a much higher frequency than camera frames, directly using the above motion dynamics requires recomputing the integrations during every optimization iteration. To improve computational efficiency, IMU preintegration has been widely adopted. This technique converts IMU measurements into relative motion increments as

$$\begin{aligned} \alpha_j^i &= \iint_{t \in [i,j]} R_t^{b_i} (\tilde{a}^{b_i} - b_a) dt^2 \\ \beta_j^i &= \int_{t \in [i,j]} R_t^{b_i} (\tilde{a}^{b_i} - b_a) dt \\ \gamma_j^i &= \int_{t \in [i,j]} \frac{1}{2} \Omega(\tilde{\omega}^{b_i} - b_\omega) \gamma_t^i dt \end{aligned} \quad (4)$$

where α_j^i , β_j^i , and γ_j^i represent the preintegrated translation, velocity, and rotation, respectively.

During preintegration, the sensor biases are assumed to remain constant. However, in visual-inertial SLAM systems, biases are treated as state variables within the optimization. When the bias estimates are updated, the preintegration terms can be efficiently relinearized using the following approximations:

$$\begin{aligned} \tilde{\alpha}_j^i &\triangleq \alpha_j^i + J_{b_a}^\alpha \delta b_a + J_{b_\omega}^\alpha \delta b_\omega \\ \tilde{\beta}_j^i &\triangleq \beta_j^i + J_{b_a}^\beta \delta b_a + J_{b_\omega}^\beta \delta b_\omega \\ \tilde{\gamma}_j^i &\triangleq \gamma_j^i \otimes \begin{bmatrix} \frac{1}{2} J_{b_\omega}^\gamma \delta b_\omega \\ 1 \end{bmatrix} \end{aligned} \quad (5)$$

where δb_a and δb_ω denote the increments of the accelerometer and gyroscope biases, respectively, and J represents the Jacobians of the preintegration terms with respect to the IMU biases. $\tilde{\alpha}_j^i$, $\tilde{\beta}_j^i$, and $\tilde{\gamma}_j^i$ are the updated preintegrated terms.

2) *Scene Representation and Rendering*: Our system adopts Gaussian splatting as the representation of the scene. Both the geometry and appearance are encoded by a set of anisotropic 3-D Gaussian primitives $\mathcal{G} = \{\mathbf{g}_i\}$. Each Gaussian primitive $\mathbf{g}_i$ is parameterized and optimized in the mapping thread to best reconstruct the observed scene

- 1) *Position (mean)*: $\mu_i \in \mathbb{R}^3$ denotes the 3-D center of the Gaussian in the world coordinate system.
- 2) *Covariance*: Represented by a positive semi-definite matrix $\Sigma_i \in \mathbb{R}^{3 \times 3}$, which is computed as $\Sigma_i = \mathbf{R}_i \mathbf{S}_i \mathbf{S}_i^T \mathbf{R}_i^T$,

where $\mathbf{S}_i$ is a diagonal scale matrix containing scale parameters (s_1, s_2, s_3) , and $\mathbf{R}_i$ is a rotation matrix. This formulation allows the Gaussian to be arbitrarily stretched and oriented.

- 3) *Opacity*: $o_i \in [0, 1]$ controls the contribution of the Gaussian to the final rendered pixel. Values close to 1 correspond to solid surface elements, whereas lower values indicate semi-transparent regions.
- 4) *Appearance (Color)*: View-dependent color effects $\mathbf{c}_i(\mathbf{v})$, where $\mathbf{v}$ denotes the viewing direction, are modeled using third-order Spherical Harmonics (SH) coefficients $\mathbf{c}_i^{\text{sh}} \in \mathbb{R}^{16}$, enabling photorealistic rendering from varying viewpoints.

This parameterization yields a compact, differentiable, and expressive model capable of capturing complex scene geometry and appearance.

Given a target camera view defined by the intrinsic matrix $\mathbf{K}$ and the pose matrix $\mathbf{P}_c = [\mathbf{R}_c | \mathbf{p}_c]$, each 3-D Gaussian is projected onto the 2-D image plane as

$$\begin{aligned} \boldsymbol{\mu}'_i &= \mathbf{K} (\mathbf{R}_c \boldsymbol{\mu}_i + \mathbf{p}_c) \\ \boldsymbol{\Sigma}'_i &= \mathbf{J} \mathbf{P}^{-1} \boldsymbol{\Sigma}_i \mathbf{P}^{-T} \mathbf{J}^T \end{aligned} \quad (6)$$

where $\mathbf{J}$ is the Jacobian of the affine approximation of the projection function. Rendering is then performed by depth-sorted alpha blending of the Gaussians within each tile

$$\hat{\mathbf{C}} = \sum_{i \in N} \mathbf{c}_i^{\text{sh}} \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \quad (7)$$

where α_i denotes the blending coefficient of each Gaussian in screen space, computed from its covariance and opacity.

Depth estimation follows a similar discrete volumetric rendering approximation

$$\hat{\mathbf{D}} = \sum_{i \in N} d_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \quad (8)$$

where d_i is the z -coordinate of the i th Gaussian in the view space.

We encourage each 3-D Gaussian to adopt a disk-like shape aligned with the surface, meaning that one scale axis becomes significantly smaller than the other two. The minimized scale axis is used as an approximation of the surface normal direction

$$\hat{\mathbf{n}}_i = \mathbf{R}_c \cdot \mathbf{e}_k \quad (9)$$

where $\mathbf{e}_k$ is the eigenvector corresponding to the smallest eigenvalue of $\boldsymbol{\Sigma}_i$. To ensure consistency, the normal direction is flipped if $\hat{\mathbf{n}}_i$ has a negative dot product with the camera viewing axis. The rendering of normal maps is then formulated as

$$\hat{\mathbf{N}} = \sum_{i \in N} \hat{\mathbf{n}}_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j). \quad (10)$$

This approach computes approximate surface normals directly from the intrinsic Gaussian attributes without introducing additional learnable parameters, thereby enhancing the geometric fidelity of the reconstruction.

B. Point-Line-IMU Initialization and Tracking

In 3DGS-SLAM, the stability of system initialization and tracking directly affects the geometric accuracy and appearance consistency of the final reconstruction. Traditional visual odometry methods that rely solely on point features often fail to provide sufficient high-quality constraints in weakly textured or repetitive environments, leading to scale ambiguity and frequent tracking loss. To alleviate this issue, several methods, such as [10], incorporate line features. On the one hand, line features are widely distributed in both indoor and outdoor structured scenes and are more robust to texture deficiency and illumination changes, thereby significantly improving front-end stability. On the other hand, line features help preserve object contours and structural boundaries in 3DGS rendering, enhancing both geometric fidelity and visual consistency. Despite these advantages, purely visual approaches still tend to fail under fast motion, motion blur, or drastic illumination variations. To address this limitation, we further integrate IMU observations, which provide high-frequency motion constraints for initialization and tracking, thereby enhancing pose estimation stability in visually degraded conditions and offering a reliable trajectory prior for subsequent Gaussian optimization.

Although introducing the IMU into 3DGS-SLAM provides high-frequency constraints for pose estimation, it also introduces unique challenges distinct from traditional SLAM. Conventional SLAM systems typically generate sparse or dense point clouds in the early mapping stage, where only 3-D geometric positions need to be explicitly estimated. By contrast, 3DGS requires implicit optimization of multiple parameters—including spatial position, scale, orientation, and appearance—resulting in significantly higher computational complexity than point cloud processing. If initialization is delayed, large-scale iterative optimization of Gaussian primitives begins before a stable global coordinate frame is established, causing frequent and substantial parameter adjustments due to drift. This not only increases computational overhead but may also introduce irreversible visual artifacts and geometric distortions into the reconstruction. Consequently, designing a fast and robust IMU initialization method that locks the global scale and pose reference as early as possible is critical for ensuring the quality of 3DGS-SLAM reconstructions.

To this end, we propose a point-line-IMU collaborative strategy for fast initialization and stable tracking. In this strategy, point and line features are extracted from RGB images using classical geometric feature extraction methods. In particular, point features are detected and described using the ORB feature [20], while line features are extracted using the EDLines [46]. During initialization, both point and line constraints are jointly exploited, where epipolar constraints from points and lines are used in a closed-form manner to directly estimate gyroscope biases. Subsequently, a hybrid analytical-optimization scheme jointly solves for accelerometer bias, scale factor, and gravity direction, achieving faster convergence under gravity constraints. This strategy rapidly establishes the global scale and pose reference at an early mapping stage, thereby reducing frequent adjustments of Gaussian parameters under unstable coordinate frames, fundamentally

lowering optimization overhead and preventing artifacts or distortions. Finally, within a sliding-window factor graph, we jointly minimize IMU residuals, point reprojection errors, and line distance residuals to achieve accurate and stable tracking. Experimental results demonstrate that the proposed method not only significantly reduces initialization time but also maintains robust tracking in weakly textured environments, laying a solid foundation for high-quality 3-D reconstruction.

1) *Point-Line Assisted IMU Initialization:* In monocular-inertial systems, the initialization process requires calibration of IMU biases, recovery of the global scale, estimation of the gravity direction, and initialization of the first set of map features. Among these, the gyroscope bias can be optimized using 2-D point and line observations across image frames, as shown in Fig. 2. For the k th pair of matched points P_i and P_j , the unit bearing vectors from the camera center to the feature locations are denoted as f_i^k and f_j^k . The corresponding epipolar plane normal is then given by $n_k = f_i^k \wedge R_{c_i}^{c_j} f_j^k$, where $R_{c_i}^{c_j}$ is the relative rotation from camera i to camera j , computed from extrinsic calibration and IMU measurements. Since all point pairs share the same baseline, the normals lie in the same plane, making the matrix $N_{ij} = [n_1, n_2, \dots, n_n]$ rank-deficient. Consequently, the smallest eigenvalue $\lambda_{M_{ij}, \min}$ of $M_{ij} = N_{ij} N_{ij}^T$ should ideally approach zero.

Image line segments can be represented using Plücker line coordinates, which provide a six-degree-of-freedom representation consisting of two 3-D vectors. In particular, l denotes the direction vector of the line, while d represents the vector from the origin to the midpoint of the line segment

$$L_i = \begin{pmatrix} l_i \\ d_i \times l_i \end{pmatrix}. \quad (11)$$

Following the line epipolar constraint introduced in [47], a generalized epipolar plane normal can be constructed as

$$g_k = \begin{pmatrix} l_i^k \wedge R_{c_i}^{c_j} l_j^k \\ l_i^k (d_i^k \wedge R_{c_i}^{c_j} - R_{c_i}^{c_j} d_i^k \wedge) l_j^k \end{pmatrix}. \quad (12)$$

Analogous to point features, we can construct $H' = [g_1, g_2, \dots, g_n]$ and $M' = H' H'^T$, where $\lambda_{M', \min}$ denotes the smallest eigenvalue of M' . By jointly considering point- and line-based constraints, the gyroscope bias can then be directly estimated by solving

$$\delta b_g^* = \arg \min_{\delta b_g} \lambda \quad (13)$$

$$\lambda = \sum_{(i,j) \in \mathcal{P}, (i',j') \in \mathcal{L}} \sqrt{\lambda_{M_{ij}, \min}} + \sqrt{\lambda_{M', \min}}$$

where the square-root formulation accelerates convergence during iterative optimization. The optimized result is subsequently used to update the rotation preintegration.

For accelerometer bias, gravity direction, and scale estimation, we adopt an analytical formulation. Given the camera-to-body extrinsic rotation R_c^b and translation t_c^b , the camera poses can be transformed into the body frame as

$$\begin{aligned} R_{b_i}^w &= R_{c_i}^w R_c^b \\ p_{b_i}^w &= s p_{c_i}^w + R_{c_i}^w t_c^b \end{aligned} \quad (14)$$

where the state variable to be optimized is $x = [\delta b_a, g, s]^T$. For n consecutive keyframe triplets (i, j, k) , translation preintegration yields residuals of the form

$$\begin{aligned} R_{b_i}^w \alpha_{b_i}^{b_j} - p_{b_i}^w + p_{b_i}^w + v_i^w \Delta t_{ij} + \frac{1}{2} g \Delta t_{ij}^2 \\ = R_{b_j}^w \alpha_{b_j}^{b_k} - p_{b_k}^w + p_{b_j}^w + v_j^w \Delta t_{jk} + \frac{1}{2} g \Delta t_{jk}^2. \end{aligned} \quad (15)$$

Assuming a constant gyroscope bias, the residual can be expressed as

$$r_j(x) = [A_j \quad B_j \quad C_j] x - D_j \quad (16)$$

where

$$\begin{aligned} A_j &= \frac{R_{c_j}^w J_{b_a, \alpha_{b_j}^{b_k}}}{\Delta t_{jk}} - \frac{R_{c_i}^w J_{b_a, \alpha_{b_i}^{b_j}}}{\Delta t_{ij}} \\ B_j &= \frac{1}{2} (\Delta t_{jk} + \Delta t_{ij}) I_{3 \times 3} \\ C_j &= \frac{p_{c_k}^w - p_{c_j}^w}{\Delta t_{jk}} - \frac{p_{c_j}^w - p_{c_i}^w}{\Delta t_{ij}} \\ D_j &= \frac{R_{b_i}^w \alpha_{b_i}^{b_j}}{\Delta t_{ij}} - \frac{R_{b_j}^w \alpha_{b_j}^{b_k}}{\Delta t_{jk}} - R_{b_j}^w \beta_{b_i}^{b_j} \\ &\quad + \left(\frac{R_{c_k}^w - R_{c_j}^w}{\Delta t_{jk}} - \frac{R_{c_j}^w - R_{c_i}^w}{\Delta t_{ij}} \right) t_b^c. \end{aligned} \quad (17)$$

This leads to a constrained optimization problem

$$\begin{aligned} x^* &= \arg \min_x \|r_p(x)\|_{\Sigma_p}^2 + \sum_{j=1}^n \|r_j(x)\|_{\Sigma_j}^2 \\ &= x^T E x + F x + G \\ \|g\| &= \text{Constant} \end{aligned} \quad (18)$$

where

$$\begin{aligned} E &= \Sigma_p^{-1} + \sum_{j=1}^n [A_j \quad B_j \quad C_j]^T \Sigma_j^{-1} [A_j \quad B_j \quad C_j] \\ F &= \sum_{j=1}^n -2 D_j^T \Sigma_j^{-1} [A_j \quad B_j \quad C_j] \\ G &= \sum_{j=1}^n D_j^T \Sigma_j^{-1} D_j \end{aligned} \quad (19)$$

and $r_p(x)$ with Σ_p denotes the prior residual and its covariance, while Σ_j is the covariance of residual errors. Under the constraint of constant gravity magnitude, the cost function can be efficiently solved using the method of Lagrange multipliers [48], which achieves faster performance compared to conventional approaches.

2) *Factor-Graph Optimization:* We employ a tightly-coupled sliding-window factor-graph optimization to obtain MAP estimates. The state vector $\mathcal{X}$ used in the system is defined as

$$\begin{aligned} \mathcal{X} &= [x_0, x_1, \dots, x_{n_k}, \lambda_0, \lambda_1, \dots, \lambda_{n_p}, o_0, o_1, \dots, o_{n_l}] \\ x &= [p_b^w, q_b^w, v_b^w, b_a, b_g] \\ o &= [\phi_1, \phi_2, \phi_3, \theta] \end{aligned} \quad (20)$$

where n_k , n_p , and n_l denote the number of keyframes, point features, and line features within the sliding window, respectively. Each IMU state x consists of the position p_b^w , orientation

q_b^w , velocity v_b^w in the world frame, as well as accelerometer bias b_a and gyroscope bias b_g . The variable o represents the orthogonal four-parameter representation of a 3-D line.

The IMU preintegration is modeled as a constraint factor connecting consecutive keyframes, with the residual formulated as

$$\mathbf{r}_I(\mathbf{z}_{b_{k+1}}^w, \mathcal{X}) = \begin{bmatrix} \delta \alpha_{b_{k+1}}^{b_k} \\ \delta \beta_{b_{k+1}}^{b_k} \\ \delta \theta_{b_{k+1}}^{b_k} \\ \delta \mathbf{b}_a \\ \delta \mathbf{b}_g \end{bmatrix} = \begin{bmatrix} \mathbf{R}_w^{b_k} \left(\mathbf{p}_{b_{k+1}}^w - \mathbf{p}_{b_k}^w + \frac{1}{2} \mathbf{g}^w \Delta t_k^2 - \mathbf{v}_{b_k}^w \Delta t_k \right) - \hat{\alpha}_{b_{k+1}}^{b_k} \\ \mathbf{R}_w^{b_k} \left(\mathbf{v}_{b_{k+1}}^w + \mathbf{g}^w \Delta t_k - \mathbf{v}_{b_k}^w \right) - \hat{\beta}_{b_{k+1}}^{b_k} \\ 2 \left[\mathbf{q}_{b_k}^{w-1} \otimes \mathbf{q}_{b_{k+1}}^w \otimes \left(\hat{\gamma}_{b_{k+1}}^{b_k} \right)^{-1} \right]_{xyz} \\ \mathbf{b}_{a_{k+1}} - \mathbf{b}_{a_k} \\ \mathbf{b}_{g_{k+1}} - \mathbf{b}_{g_k} \end{bmatrix}. \quad (21)$$

For point features, the reprojection error is used as the residual term

$$\mathbf{r}_P(\mathbf{z}_{p_i}^{c_k}, \mathcal{X}) = [\mathbf{b}_1 \mathbf{b}_2]^\top \cdot \left(\mathcal{P}_{p_i}^{c_k} - \frac{\hat{\mathcal{P}}_{p_i}^{c_k}}{\|\hat{\mathcal{P}}_{p_i}^{c_k}\|} \right) \\ \mathcal{P}_{p_i}^{c_k} = \pi_c^{-1} \left(\begin{bmatrix} u_{p_i}^{c_k} \\ v_{p_i}^{c_k} \end{bmatrix} \right) \\ \hat{\mathcal{P}}_{p_i}^{c_k} = \mathbf{R}_b^{c_k} \left(\mathbf{R}_{b_j}^{b_k} \left(\mathbf{R}_c^{b_k} \frac{1}{\lambda_{p_i}} \pi_c^{-1} \left(\begin{bmatrix} u_{p_i}^{c_j} \\ v_{p_i}^{c_j} \end{bmatrix} \right) + \mathbf{p}_c^b \right) + \mathbf{p}_{b_j}^w - \mathbf{p}_{b_k}^w \right) - \mathbf{p}_c^b. \quad (22)$$

Here, $[u_{p_i}^{c_j}, v_{p_i}^{c_j}]$ is the first observation of point p_i in frame j , while $[u_{p_i}^{c_k}, v_{p_i}^{c_k}]$ is the observation in frame k . The coordinates $\mathcal{P}_{p_i}^{c_k}$ are directly computed from the projection matrix π and the observation at time k , while $\hat{\mathcal{P}}_{p_i}^{c_k}$ is estimated using the state variables and the observation from frame j .

For line features, the residual is defined as the distance between the midpoint $\mathbf{m}$ of the observed segment and the projected line $\mathbf{l}$

$$\mathbf{r}_L(\mathbf{z}_{l_i}^{c_k}, \mathcal{X}) = d(\mathbf{m}, \mathbf{l}) = \frac{\mathbf{m}^\top \mathbf{l}}{l_1^2 + l_2^2}. \quad (23)$$

Given a 3-D line $\mathcal{L}_i$ with Plücker coordinates l_i^w in the world frame, it can be transformed into the k th camera frame as

$$l_i^{c_k} = \begin{bmatrix} n_c \\ d_c \end{bmatrix} l_i^w = \begin{bmatrix} R_w^{c_k} & [p_w^{c_k}]^\wedge R_w^{c_k} \\ 0 & R_w^{c_k} \end{bmatrix} l_i^w \quad (24)$$

where $p_w^{c_k}$ and $R_w^{c_k}$ denote the translation and rotation, respectively. The corresponding projected line on the image plane is then given by

$$\mathbf{l} = [l_1, l_2, l_3]^\top = \mathbf{K} n_c \quad (25)$$

where $\mathbf{K}$ is the line projection matrix, and n_c is extracted from 24.

All the above measurement residuals jointly form the overall objective function, which minimizes the errors of all observations

$$\min_{\mathcal{X}} \left\{ \|\mathbf{r}_0 - \mathbf{J}_0 \mathcal{X}\|^2 + \sum_{k \in \mathcal{B}} \left\| \mathbf{r}_I(\mathbf{z}_{b_{k+1}}^{b_k}, \mathcal{X}) \right\|_{\Sigma_{b_{k+1}}^{b_k}}^2 + \sum_{(k,i) \in \mathcal{P}} \rho_P \left\| \mathbf{r}_P(\mathbf{z}_{p_i}^{c_k}, \mathcal{X}) \right\|_{\Sigma_{p_i}^{c_k}}^2 + \sum_{(k,i) \in \mathcal{L}} \rho_L \left\| \mathbf{r}_L(\mathbf{z}_{l_i}^{c_k}, \mathcal{X}) \right\|_{\Sigma_{l_i}^{c_k}}^2 \right\} \quad (26)$$

where Σ denotes the covariance matrix associated with each type of residual, reflecting measurement uncertainty and ensuring that more reliable observations contribute more strongly to the optimization. ρ denotes the robust loss function applied to each residual term; here, we adopt the Huber norm to mitigate the influence of outliers by down-weighting large residuals.

C. Geometry-Aware Density Optimization

Despite recent advances, existing 3DGS-SLAM frameworks fused with IMU measurements still face limitations. On the one hand, IMU integration remains insufficiently exploited. While the IMU provides high-frequency and stable motion information that enhances pose estimation and trajectory tracking, its role is often restricted to the tracking stage and fails to provide support for mapping. On the other hand, the optimization of Gaussian splatting heavily depends on the availability of large-scale, high-quality initial point clouds. However, sparse-feature-based VIO methods can only provide a limited number of feature points. Insufficient or unevenly distributed initial points inevitably degrade the quality of Gaussian-based mapping. This issue is particularly pronounced in structured, weakly textured environments, where the scarcity of features and frequent mismatches further exacerbate mapping deterioration.

To address these challenges, we propose a geometry-aware density optimization framework, which resolves the above issues through three key steps. First, IMU dynamics constraints are introduced to achieve metric-scaled depth estimation, thereby overcoming the scale ambiguity inherent in traditional unsupervised approaches and providing a reliable depth prior for Gaussian splatting optimization. Second, depth maps, normal maps, and line features are jointly utilized to construct planar hypotheses, from which structural planes are extracted as core geometric References to guide Gaussian optimization. Third, based on the detected planes, feature pruning and point cloud densification are performed. Erroneous correspondences (“outlier points”) are removed by enforcing planar geometric consistency, while the point density in weakly textured regions is adaptively increased under planar guidance. This process supplies Gaussian splatting with a more accurate and denser initialization set.

By explicitly establishing the link between the point cloud and scene geometry, the proposed method significantly improves both the geometric accuracy and spatial density of the point cloud, thereby enhancing mapping performance, especially in weakly textured and structured environments.

1) *Metric-Scale Depth Estimation*: Conventional unsupervised monocular depth estimation suffers from scale ambiguity, as the photometric loss based on inverse warping enforces only relative geometric consistency but lacks absolute metric constraints. To overcome this limitation, Zhang et al. [19] introduced IMU dynamics to impose absolute scale constraints. In particular, their framework incorporates an IMU photometric loss, which directly associates depth predictions with absolute motion scales. The formulation is defined as

$$L_{\text{photo}}^{\text{IMU}} = \frac{1}{N} \sum_{i=1}^N \min_{\delta} \mathcal{L} [I(y_i), I_{\delta}(\phi(\mathcal{B}(y_i)))]$$

$$\mathcal{B}_{\delta}(y) = KR_{\delta}K^{-1}y + \frac{Kp_{\delta}}{\hat{z}}$$

$$\mathcal{L}[I, I_{\delta}] = \alpha \frac{1 - \text{SSIM}(I, I_{\delta})}{2} + (1 - \alpha) \|I - I_{\delta}\|_1 \quad (27)$$

where $\mathcal{B}$ is the back-projection function that projects a pixel y in the reference keyframe I to the corresponding coordinate in the adjacent keyframe I_{δ} ($\delta = \pm 1$). Here, K denotes the camera intrinsics, $\hat{z}$ is the estimated depth of pixel y , and (R_{δ}, p_{δ}) are the relative rotation and translation from I to I_{δ} , obtained from factor-graph optimization. N is the total number of pixels in the image, and $\phi(\cdot)$ denotes a depth normalization function. The photometric similarity function $\mathcal{L}$ combines the structural similarity index (SSIM) and the $L1$ norm to measure the appearance consistency between corresponding pixels.

To further align ego-motion estimated by the depth network with that obtained from IMU integration, following Zhang et al. [19], we adopt a cross-sensor photometric consistency loss:

$$L_{\text{photo}}^{\text{cons}} = \frac{1}{N} \sum_{i=1}^N \min_{\delta} \mathcal{L} [I_{\delta}(\phi(\mathcal{B}_{\delta}(y_i))), I_{\delta}(\phi(\hat{\mathcal{B}}_{\delta}(y_i)))] \quad (28)$$

where $\hat{\mathcal{B}}$ is defined analogously to $\mathcal{B}$, except that the transformation is derived from the depth network prediction rather than IMU measurements.

Finally, the total training loss integrates these IMU-related terms with vision-based constraints, expressed as

$$L_{\text{total}} = L_{\text{photo}}^{\text{vis}} + \lambda_1 L_s + \lambda_2 L_{\text{photo}}^{\text{IMU}} + \lambda_3 L_{\text{photo}}^{\text{cons}} \quad (29)$$

where $L_{\text{photo}}^{\text{vis}}$ and L_s denote the standard photometric and smoothness losses, respectively, while $\lambda_1, \lambda_2, \lambda_3$ are balancing weights. While IMU supervision enables metric-scale depth prediction, such depth maps alone are still noisy and locally inconsistent, and cannot be directly used to reliably initialize or constrain Gaussian primitives, motivating the geometry-aware integration described in Sections III-C2 and III-C3.

2) *Planar Hypothesis Construction*: We propose a plane hypothesis construction method that integrates depth, line features, and normal maps. Our method does not rely on any structural assumptions. Instead, planar structures are detected purely based on local geometric consistency in depth and surface normals, allowing planes with arbitrary orientations to be robustly recovered in both structured and unstructured scenes, as shown in Fig. 3. The detailed procedure is summarized in Algorithm 1.

The process begins by computing a normal map from the estimated depth map. For each pixel $p(u, v)$, the normal vector

Algorithm 1 Planar Hypothesis Construction

Require: Depth map $\mathcal{D}$, Line features $\mathcal{L}$

Ensure: Planar segments $\mathcal{P}$, Plane equations $\mathcal{F}$

```

1  $\mathcal{N} \leftarrow \text{ComputeNormals}(\mathcal{D})$ ,  $\mathcal{V} \leftarrow \mathbf{0}_{H \times W}$ 
2 for each unvisited pixel  $p \notin \mathcal{L}$  do
3   init region  $\mathcal{P}_{\text{temp}} = \{p\}$ , push  $p$  to  $\mathcal{Q}$ , set  $\mathcal{V}[p] = 1$ ,
    $\mathbf{n}_{\text{mean}} = \mathcal{N}[p]$ 
4   while  $\mathcal{Q} \neq \emptyset$  do
5      $q \leftarrow \mathcal{Q}.\text{pop}()$ 
6     for each neighbor  $r$  of  $q$  where  $\mathcal{V}[r] = 0 \wedge \mathcal{L}[r] = 0$ 
       do
7       if  $\cos^{-1}(|\mathbf{n}_{\text{mean}} \cdot \mathcal{N}[r]|) < \theta_{\text{ang}} \wedge |\mathcal{D}[q] - \mathcal{D}[r]| < \theta_{\text{dep}}$  then
8         add  $r$  to  $\mathcal{P}_{\text{temp}}$ , set  $\mathcal{V}[r] = 1$ , push  $r$  to  $\mathcal{Q}$ 
9       end if
10    end for
11     $\mathbf{n}_{\text{avg}} \leftarrow \text{MeanShift}(\mathcal{N}(\mathcal{P}_{\text{temp}}))$ 
12  end while
13  if  $|\mathcal{P}_{\text{temp}}| > \tau_{\text{size}}$  then
14     $\mathcal{P}.\text{add}(\mathcal{P}_{\text{temp}})$ 
15  end if
16 end for
17 for each  $\mathcal{P}_i \in \mathcal{P}$  do
18    $\mathbf{X}_i = \{\pi^{-1}(u, v, \mathcal{D}[u, v]) \mid (u, v) \in \mathcal{P}_i\}$ ,  $(\mathbf{n}_i, d_i) \leftarrow \text{RANSAC}(\mathbf{X}_i, \rho)$ 
19    $\mathcal{F}.\text{add}((\mathbf{n}_i, d_i))$ 
20 end for
21  $\text{MergePlanes}(\mathcal{P})$ 
22  $\text{CompleteBoundaries}(\mathcal{P}, \mathcal{L}, \mathcal{D}, \theta_{\text{dep}})$ 
23  $\text{FilterByArea}(\mathcal{P}, \mathcal{F}, \tau_{\text{large}})$ 
```

is obtained by normalizing the cross product of the depth gradients along the u - and v -axes

$$\mathbf{n}(u, v) = \frac{\nabla_u p \times \nabla_v p}{|\nabla_u p \times \nabla_v p|} \quad (30)$$

where ∇_u and ∇_v denote the derivatives of the depth map along u and v , respectively. A 3×3 median filter is then applied to suppress noise. Compared with raw depth values, surface normals provide stronger robustness for plane detection while also serving as geometric supervision for Gaussian mapping.

Next, planar region growing is performed. After initializing a visited mask, each unvisited pixel that does not overlap with line features is treated as a seed point. From each seed, an 8-neighborhood breadth-first search (BFS) is conducted under three constraints: 1) the angular difference between the candidate pixel normal and the region's mean normal must be smaller than θ_{ang} ; 2) the depth difference with the current pixel must be below the adaptive threshold $\theta_{\text{dep}} = \lambda_{\text{dep}} \times \text{median}(\mathcal{D})$; and 3) the region cannot grow across line features. This adaptive thresholding strategy enables the plane growing process to automatically adjust to scene scale and depth distribution, improving robustness in weakly structured and nonstructured environments. To improve robustness against local geometric variations, the mean normal is dynamically updated during growth using mean-shift clustering on the unit sphere [49]. Regions with fewer than τ_{size} pixels are discarded as fragments.

For each resulting region $\mathcal{P}_i$, the pixels are back-projected into 3-D space to form a point set $\mathbf{X}_i$. Plane parameters are robustly estimated via RANSAC with a Huber loss

$$\min_{\mathbf{n}, d} \sum \rho(|\mathbf{n}^\top \mathbf{x} + d|) \quad (31)$$

where $\mathbf{n}$ is the plane normal and d the plane offset. To address fragmentation caused by depth estimation errors, adjacent planes are iteratively merged if they satisfy

$$\text{Merge}(\mathcal{P}_i, \mathcal{P}_j) \Leftarrow \begin{cases} \cos^{-1}(|\mathbf{n}_i \cdot \mathbf{n}_j|) < \theta_{\text{ang}} \\ \min_{\mathbf{x} \in \mathbf{X}_i} |\mathbf{n}_j^\top \mathbf{x} + d_j| < \theta_{\text{dist}} \\ \text{Adjacent}(\mathcal{P}_i, \mathcal{P}_j). \end{cases} \quad (32)$$

Once merged, the plane equation is re-estimated. This process is repeated until no further merges are possible.

Finally, boundary completion is performed. For regions not delineated by line features, linear boundary segments are fit to points exhibiting significant depth discontinuity ($|\nabla D| > \theta_{\text{dep}}$). Planes are then filtered by pixel area, retaining only those larger than τ_{large} . The resulting planar hypotheses provide accurate segmentation of dominant structural surfaces, offering reliable geometric constraints for subsequent mapping.

3) *Geometry-Driven Densification*: In low-texture environments, image features are often indistinct, leading to erroneous correspondences and floating artifacts in the Gaussian representation. Moreover, the number of initial points generated by feature-based triangulation is inherently limited. To address these issues, we propose a geometry-driven algorithm that leverages planar structures for Gaussian pruning and densification. This approach alleviates the computational burden of subsequent Gaussian mapping while improving both real-time performance and reconstruction accuracy.

The algorithm begins by pruning mismatched feature points through structured plane verification. Feature points lying on planar hypotheses are first extracted from RGB images and then projected onto their corresponding planes. Points with a distance greater than a preset threshold or projected outside the plane boundary are identified as mismatches and discarded. The 3-D coordinates of the remaining points are then corrected to lie precisely on the plane surface

$$\mathbf{P}' = \mathbf{P} - [(\mathbf{P} - \mathbf{P}_0) \cdot \mathbf{n}] \mathbf{n} \quad (33)$$

where $\mathbf{P}'$ denotes the corrected coordinates of a feature point, $\mathbf{P}$ its original coordinates, and $\mathbf{P}_0$ a reference point on the plane.

Next, point cloud densification is performed. To ensure structural completeness and sharpness of planar edges, edge sampling is carried out. A plane boundary is represented by a set of vertices $V = \{v_1, v_2, \dots, v_m\}$ and line segments $e_i = \langle v_i, v_{i+1} \rangle$. For each segment e_i , with length $L_i = \|v_{i+1} - v_i\|_2$, the number of sampled points is determined as $n_i = \max(2, \lceil (L_i)/(s_0) \rceil)$, where s_0 denotes the sampling interval. The $\max(2, \cdot)$ term guarantees that both endpoints are sampled. The sample points are generated by linear interpolation

$$p_{i,j} = v_i + \frac{j}{n_i - 1} (v_{i+1} - v_i), \quad j = 0, 1, \dots, n_i - 1. \quad (34)$$

Subsequently, in-plane densification is performed. A base sampling interval d_0 , dynamically adjusted according to image resolution, is used to generate a grid of candidate points within the plane. Each candidate is further evaluated using both feature point density and texture complexity. In particular, for a candidate point (u, v) , the local feature density factor is defined as

$$\alpha(u, v) = \exp\left(-\frac{N(u, v)}{r^2 \cdot \rho_0}\right) \quad (35)$$

where $N(u, v)$ is the number of feature points within a square window of diameter r centered at (u, v) , and ρ_0 is the expected feature density derived from the sampling interval and plane area.

The texture complexity factor is computed as

$$\beta(u, v) = \frac{1}{1 + \exp(-k \cdot (S(u, v) - T))} \quad (36)$$

$$S(u, v) = \sum_{(x,y) \in W(u,v)} (\nabla_x^2(x, y) + \nabla_y^2(x, y))$$

where $W(u, v)$ is a local window centered at (u, v) , ∇_x and ∇_y denote the image gradients in the x - and y -directions, k is a scaling factor, and T is the texture complexity threshold.

Finally, candidate points are selected based on a joint criterion that considers both local feature density and texture complexity, ensuring adaptive densification in regions with insufficient texture. By incorporating geometric constraints, the proposed method improves the accuracy of initial point placement and adaptively increases point cloud density, thereby enabling robust reconstruction even in low-texture areas.

D. Photorealistic Mapping

Although existing reconstruction methods based on 3-D Gaussian splatting can produce realistic appearances in richly textured natural scenes, they often perform poorly in structured environments with weak textures. Direct optimization at high resolution is prone to being trapped in local minima due to sparse texture regions, while also leading to low training efficiency and unstable convergence. Moreover, conventional Gaussian primitives typically maintain an approximately spherical shape, which makes it difficult to accurately fit real geometry on regular surfaces or large planar regions, often resulting in surface jitter and loss of details. In addition, existing approaches pay insufficient attention to critical geometric features such as lines and planes, which often serve as essential constraints in weakly textured regions.

To address these issues, we propose a Gaussian optimization framework that combines pyramid-based progressive optimization with diversified loss functions. In particular, we first introduce a multiscale optimization strategy based on an image pyramid. The optimization proceeds progressively from low resolution to high resolution, transitioning from global geometric alignment to local detail refinement. This approach ensures global convergence while significantly improving training efficiency and reconstruction accuracy. Second, we design a rendering loss that incorporates color, depth, and normal information to constrain the consistency between Gaussian

rendering results and observations across multiple dimensions. In addition, a shape loss encourages Gaussian primitives to evolve into disk-like forms, thereby better fitting real surfaces. Furthermore, we introduce a geometric consistency loss targeting line and plane features, which enforces the spatial alignment between Gaussian primitives and 3-D features, thus strengthening geometric constraints in weakly textured areas. By combining progressive pyramid optimization with multi-dimensional loss design, the proposed framework effectively enhances both the fitting accuracy and geometric plausibility of Gaussian splatting in photorealistic mapping tasks.

1) *Pyramid-Based Gaussian Optimization*: We employ a coarse-to-fine pyramid optimization strategy with N levels to efficiently optimize Gaussian parameters while balancing reconstruction accuracy and computational stability. Let $n \in 1, \dots, N$ denote the pyramid level, where $n = 1$ corresponds to the lowest resolution and $n = N$ corresponds to the full resolution.

To explicitly control the balance between global geometric consistency and local appearance fidelity across pyramid levels, we dynamically adjust the weights of different loss terms. In particular, for each loss term $k \in d, n, s, l, p$, the weight at pyramid level n is defined relative to that at the finest level N as

$$\lambda_k^{(n)} = \lambda_k^{(N)} \cdot \gamma_k^{(N-n)} \quad (37)$$

where γ_k is a fixed scaling factor associated with the loss term k .

In our design, geometric constraints related to line and plane features are emphasized more strongly at coarser levels by adopting larger scaling factors, i.e., $\gamma_l = \gamma_p = 2.0$. This encourages Gaussians to closely adhere to dominant planar or linear structures, thereby stabilizing the global scene layout during early optimization stages. In contrast, the depth consistency, normal consistency, and shape regularization terms use milder scaling factors, i.e., $\gamma_d = \gamma_n = \gamma_s = 1.2$, allowing them to play a supportive role without overwhelming the optimization.

As a result, at low-resolution pyramid levels, the optimization is dominated by strong geometric regularization and structural constraints, ensuring robust global alignment. As the resolution increases, all auxiliary geometric losses are progressively weakened, preventing overly restrictive constraints from hindering fine-scale appearance modeling. Consequently, at higher resolutions, the optimization naturally shifts its focus toward photometric consistency and detailed visual fidelity, enabling accurate reconstruction of local geometry and realistic surface appearance.

To maintain real-time performance and prevent computational overhead caused by an excessive number of Gaussians, we introduce a pruning strategy based on opacity and gradient magnitudes. A Gaussian is removed if both its opacity and gradient are small, indicating a limited contribution to scene representation. The pruning condition is defined as

$$\Sigma_i = \begin{cases} \Sigma_i, & \text{if } |\alpha_i| \geq |Q_\alpha(y_{\text{iter}})| \text{ and } |\nabla \Sigma_i| \geq |Q_{\Sigma_i}(y_{\text{iter}})| \\ 0, & \text{otherwise} \end{cases} \quad (38)$$

where $Q(\cdot)$ denotes the quantile function, $\nabla \Sigma_i$ is the gradient of the Gaussian primitive g_i , and y_{iter} represents the ratio

of Gaussians to be pruned. Note that y_{iter} is not a fixed constant but is dynamically adjusted according to the optimization stage. This pruning strategy effectively removes noisy or redundant Gaussians, reducing computational costs while preserving scene representation accuracy.

2) *Mapping Loss*: Gaussian primitives are optimized using selected keyframes from the tracking system, together with depth and normal maps estimated by the neural network. In this work, we divide the mapping loss into two components: rendering loss and shape loss. The rendering loss constrains the consistency between Gaussian rendering results and reference data, and it consists of color, depth, and normal terms. The color loss measures the difference between rendered colors and observed keyframes, defined as

$$L_{\text{color}} = |\hat{\mathbf{C}} - \mathbf{C}^{gt}| \quad (39)$$

while the depth loss enforces alignment between the rendered depth and the estimated depth

$$L_{\text{depth}} = |\hat{\mathbf{D}} - \mathbf{D}| \quad (40)$$

where $\mathbf{D}$ denotes the estimated depth corresponding to the keyframe $\mathbf{C}^{gt}$. To further enhance geometric consistency and regulate surface orientation, we adopt a normal loss inspired by [50]

$$L_{\text{normal}} = \sum_i \omega_i (1 - \hat{n}_i^T \mathbf{N}) \quad (41)$$

where ω_i represents blending weights of intersections, and $\mathbf{N}$ is derived by analytically differentiating the depth estimation.

For shape regularization, we minimize the smallest eigenvalue of the covariance matrix of Gaussian primitives during training, encouraging them to take disk-like forms that better fit scene surfaces

$$L_{\text{scale}} = \sum_{g_i \in \mathcal{G}} \lambda_{\min}^i \quad (42)$$

where $\lambda_{\min}^i$ denotes the smallest eigenvalue of the covariance matrix Σ_i of Gaussian primitive g_i .

To further enforce spatial consistency, we introduce geometric consistency losses for line and plane features. For Gaussians generated from line features e_j , we constrain their centers μ_i to lie close to the line direction. Let a line be defined by endpoints v_1, v_2 with direction vector $\mathbf{d} = v_2 - v_1$, then the minimum distance between μ_i and line e_j is

$$\text{dist}(\mu_i, e_j) = \frac{|(\mu_i - v_1) \times \mathbf{d}|}{|\mathbf{d}|}. \quad (43)$$

Accordingly, the line consistency loss is defined as

$$L_{\text{line}} = \frac{1}{N_l} \sum_{\mu_i \in e_j} \left(\frac{|(\mu_i - v_1) \times \mathbf{d}|}{|\mathbf{d}|} \right)^2. \quad (44)$$

For Gaussians generated from planes in Section III-C3, their centers μ_i should lie on the plane as much as possible, leading to the following plane consistency loss:

$$L_{\text{plane}} = \frac{1}{N_p} \sum_{\mu_i \in \mathcal{P}} (\mathbf{n}^\top \mu_i + d)^2 \quad (45)$$

where $\mathbf{n}$ and d denote the normal vector and offset of the plane, respectively.

Combining rendering, geometric, and shape losses, the final mapping loss can be written as

$$L_{\text{mapping}} = L_{\text{color}} + \lambda_d L_{\text{depth}} + \lambda_n L_{\text{normal}} + \lambda_s L_{\text{scale}} + \lambda_p L_{\text{plane}} + \lambda_l L_{\text{line}} \quad (46)$$

where $\lambda_d, \lambda_n, \lambda_s, \lambda_p, \lambda_l$ are weighting parameters that balance the contributions of each loss term.

IV. EXPERIMENTS

A. Experiment Setup

1) *Datasets*: To validate the effectiveness of the proposed method in structured indoor scenes, we conducted experiments on four widely used indoor datasets: Replica [51], TUM RGB-D [52], ScanNet [53], OpenLORIS [54], and KITTI [55]. These datasets cover a wide range of application scenarios, from high-fidelity synthetic environments to large-scale real-world environments. Replica provides accurate geometry and ground-truth poses, making it suitable for testing algorithmic accuracy under ideal conditions. TUM RGB-D is a small-scale, real indoor dataset with poses precisely obtained from an external motion capture system, which allows evaluation of tracking performance in real-world scenarios. ScanNet is larger in scale and more diverse in scene types, enabling assessment of the generalization ability of the method. Since these three datasets do not provide raw IMU measurements, we follow [56] to synthesize continuous-time virtual inertial observations from their ground-truth data, ensuring consistency across experiments. OpenLORIS, on the other hand, is collected in real environments, containing challenging conditions such as low-texture regions, and provides real IMU data, making it more representative of practical applications. KITTI is a large-scale outdoor dataset collected from a vehicle platform, providing synchronized stereo/RGB images, accurate ground-truth poses, and real IMU measurements.

2) *Baselines*: To comprehensively evaluate the performance of the proposed method, we select two categories of representative baselines. The first category includes NeRF-based SLAM methods, such as Co-SLAM [33], ESLAM [34], and GO-SLAM [32], which are important for integrating neural radiance fields with camera pose tracking. The second category includes methods based on 3-D Gaussian Splatting, namely MonoGS [8], SplatAM [9], Gaussian-SLAM [39], and PhotoSLAM [7], which represent typical applications of Gaussian splatting in SLAM tasks. Together, these two categories cover the mainstream research directions and provide sufficient and representative baselines for comparison.

3) *Metrics*: We adopt multiple metrics for evaluation. For camera pose estimation, the ATE RMSE of keyframes is used to measure tracking accuracy. For RGB rendering quality, PSNR, SSIM, and LPIPS are computed every five frames to assess the similarity between reconstructed scenes and real images. For depth estimation, we use Depth L1, which measures the mean absolute error between the rendered depth and the ground-truth depth over valid pixels. It is defined as

$$\text{Depth L1} = \frac{1}{|\Omega|} \sum_{p \in \Omega} |\hat{D}(p) - D^{gt}(p)| \quad (47)$$

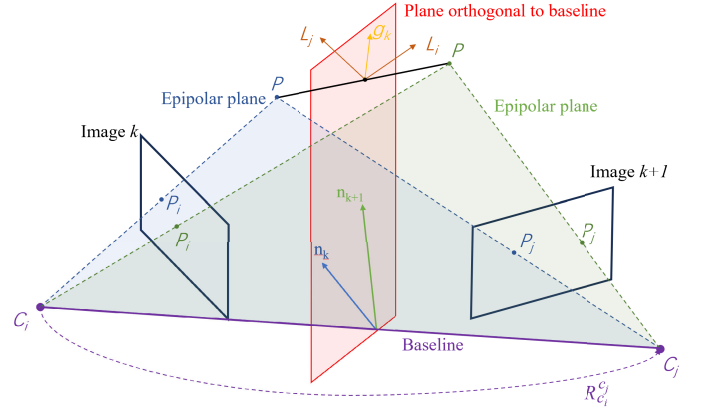


Fig. 2. Point-line constraints for gyroscope bias estimation.

where $\hat{D}(p)$ and $D^{gt}(p)$ denote the rendered depth and the ground-truth depth at pixel p , respectively, and Ω is the set of pixels where a valid ground-truth depth is available.

4) *Implementation Details*: All experiments are conducted on a laptop equipped with an Intel Core i9-13980HX CPU, an NVIDIA GeForce RTX 4090 Laptop GPU, and 32 GB of memory. All baseline methods are run with their default parameters to ensure fairness. Each experiment is repeated five times on the same data sequence, and the average result is reported to mitigate the influence of randomness. For the IMU-supervised depth estimation model, the loss weights are set to $\lambda_1 = 0.001$, $\lambda_2 = 0.5$, and $\lambda_3 = 0.01$. Regarding hyperparameter settings, the loss weights at the finest pyramid level N are fixed throughout all experiments. In particular, we set $(\lambda_d^{(N)}, \lambda_n^{(N)}, \lambda_s^{(N)}, \lambda_p^{(N)}, \lambda_l^{(N)})$ to $(0.5, 0.3, 0.2, 0.15, 0.1)$, respectively.

B. Evaluation of Tracking and Mapping

1) *Evaluation on Replica*: To evaluate the performance of our method under ideal conditions, we conducted experiments on the high-fidelity synthetic Replica dataset. As shown in Table I, our approach achieves the best ATE in most sequences, indicating more stable and accurate camera tracking. In terms of rendering quality, our method consistently outperforms others on PSNR, SSIM, and LPIPS, demonstrating that the reconstructed scenes are sharper, more photorealistic, and less distorted in detail. For depth estimation, our Depth L1 error is also significantly lower, further validating the effectiveness of the planar constraints and geometric consistency loss in maintaining geometric accuracy.

Unlike most existing methods that are evaluated only on relatively simple scenes, we further tested our method on large-scale and structurally complex environments such as an apartment and f1rl, which span across two floors. Results show that in these more challenging sequences, other methods often suffer from tracking drift or reconstruction degradation, whereas our method maintains stable pose estimation and high rendering quality, further highlighting its robustness and generalization ability.

In addition, Fig. 4 presents qualitative comparisons of reconstruction results on Replica. From the visualizations, it

TABLE I

EVALUATION OF OUR METHOD AND BASELINES ON REPLICA IN TERMS OF ACCURACY AND RENDERING QUALITY (ATE IS REPORTED IN CM). BEST RESULTS ARE INDICATED AS **FIRST**, **SECOND**, AND **THIRD**

	Methods	Metrics	room0	room1	room2	office0	office1	office2	office3	office4	apart0	apart1	apart2	frl0	frl4	Avg.
NeRF-based	ESLAM (RGB-D)	ATE↓	0.71	0.70	0.52	0.57	0.55	0.58	0.72	0.63	9.22	7.09	7.01	3.33	1.07	2.52
		PSNR↑	26.96	28.98	29.80	35.04	33.81	30.08	30.01	31.14	28.94	26.47	25.31	27.86	28.41	29.45
		SSIM↑	0.821	0.837	0.843	0.902	0.873	0.865	0.881	0.886	0.872	0.845	0.838	0.882	0.889	0.864
		LPIPS↓	0.171	0.173	0.187	0.172	0.181	0.186	0.172	0.174	0.176	0.209	0.221	0.181	0.174	0.183
	Co-SLAM (RGB-D)	Depth L1↓	0.97	1.07	1.28	0.86	1.26	1.71	1.43	1.06	1.14	1.26	1.31	1.02	0.94	1.18
		ATE↓	0.67	0.64	0.49	0.56	0.56	0.55	0.67	0.58	5.30	4.78	6.31	2.01	2.30	1.96
		PSNR↑	28.88	28.51	29.37	35.44	34.63	26.56	28.79	32.16	30.10	22.86	23.29	23.52	25.33	28.42
		SSIM↑	0.892	0.843	0.851	0.854	0.826	0.814	0.866	0.856	0.905	0.766	0.771	0.822	0.814	0.837
	GO-SLAM (RGB-D)	LPIPS↓	0.213	0.205	0.215	0.177	0.181	0.172	0.163	0.176	0.321	0.440	0.462	0.367	0.461	0.273
		Depth L1↓	1.05	0.85	2.37	1.24	1.48	1.86	1.66	1.54	3.76	4.01	3.96	4.27	2.97	2.39
		ATE↓	0.61	0.59	0.46	0.52	0.53	0.51	0.62	1.54	29.81	17.43	8.99	11.77	14.42	6.68
		PSNR↑	29.64	29.08	29.68	35.91	34.97	27.12	29.16	32.59	9.79	15.93	14.37	20.09	19.78	25.24
3DGS-based	SplaTAM (RGB-D)	SSIM↑	0.904	0.856	0.863	0.907	0.866	0.874	0.889	0.893	0.366	0.693	0.685	0.803	0.814	0.801
		LPIPS↓	0.166	0.168	0.182	0.169	0.178	0.169	0.158	0.171	0.797	0.601	0.623	0.433	0.451	0.328
		Depth L1↓	0.92	0.83	1.21	0.81	1.20	1.63	1.36	1.01	8.83	3.46	4.22	3.93	3.38	2.52
	Gaussian-SLAM (RGB-D)	ATE↓	0.31	0.40	0.29	0.47	0.27	0.29	0.32	0.55	20.33	10.97	8.22	6.03	10.49	4.53
		PSNR↑	32.49	33.72	34.65	38.29	39.04	31.91	30.05	31.83	13.12	24.57	25.52	30.72	31.86	30.60
		SSIM↑	0.975	0.970	0.980	0.982	0.982	0.965	0.952	0.929	0.415	0.821	0.858	0.924	0.930	0.900
		LPIPS↓	0.072	0.096	0.078	0.086	0.093	0.100	0.110	0.150	0.656	0.302	0.258	0.201	0.198	0.185
	Photo-SLAM (RGB-D)	Depth L1↓	0.43	0.38	0.54	0.44	0.66	1.05	1.60	0.68	4.74	3.93	3.97	2.93	2.14	1.81
		ATE↓	0.29	0.29	0.22	0.37	0.23	0.41	0.30	0.35	18.97	10.31	7.93	7.74	12.88	4.64
		PSNR↑	29.57	31.61	33.46	38.39	39.62	32.91	33.62	34.26	27.07	24.93	24.34	25.19	26.70	30.90
		SSIM↑	0.944	0.952	0.973	0.985	0.991	0.974	0.982	0.979	0.864	0.850	0.828	0.836	0.839	0.923
	MonoGS (RGB-D)	LPIPS↓	0.197	0.184	0.148	0.099	0.097	0.158	0.123	0.138	0.345	0.381	0.410	0.358	0.343	0.229
		Depth L1↓	0.61	0.25	0.54	0.50	0.52	0.98	1.63	0.42	4.31	4.18	3.63	4.61	3.97	2.01
		ATE↓	0.31	0.24	0.27	0.34	0.21	0.24	0.14	0.76	5.39	7.98	6.31	1.09	1.27	1.89
		PSNR↑	30.71	33.51	35.02	38.47	39.08	33.03	33.78	36.02	29.07	22.73	24.59	34.16	33.36	32.58
	Ours (Mono+IMU)	SSIM↑	0.899	0.934	0.951	0.964	0.961	0.938	0.952	0.922	0.796	0.848	0.940	0.932	0.921	0.921
		LPIPS↓	0.075	0.057	0.043	0.050	0.047	0.077	0.066	0.054	0.227	0.293	0.354	0.115	0.129	0.122
		Depth L1↓	0.63	0.51	0.66	0.52	0.58	0.97	1.34	0.60	4.25	3.51	2.78	1.97	1.35	1.51
		ATE↓	0.33	0.22	0.29	0.36	0.19	0.25	0.12	0.81	19.02	9.37	7.52	3.01	9.23	3.90
	Ours (Mono+IMU)	PSNR↑	34.87	36.43	37.49	39.95	42.09	36.24	36.70	36.07	21.89	26.87	27.92	30.70	26.97	33.40
		SSIM↑	0.954	0.959	0.965	0.971	0.977	0.964	0.963	0.957	0.864	0.856	0.873	0.886	0.863	0.927
		LPIPS↓	0.068	0.076	0.075	0.072	0.055	0.078	0.065	0.099	0.397	0.285	0.273	0.225	0.247	0.155
		Depth L1↓	0.79	0.56	0.91	0.70	1.10	0.95	0.97	1.66	4.57	3.66	3.32	2.94	2.88	1.92
	Ours (Mono+IMU)	ATE↓	0.29	0.21	0.24	0.32	0.20	0.23	0.16	0.63	5.29	7.74	6.44	1.17	1.09	1.85
		PSNR↑	32.41	35.32	36.27	39.58	41.09	35.46	35.17	35.95	29.22	24.55	26.07	33.45	32.72	33.64
		SSIM↑	0.927	0.953	0.976	0.982	0.986	0.957	0.965	0.961	0.933	0.814	0.869	0.941	0.929	0.937
		LPIPS↓	0.066	0.054	0.049	0.047	0.044	0.073	0.073	0.071	0.214	0.276	0.297	0.097	0.121	0.114
		Depth L1↓	0.58	0.49	0.62	0.48	0.52	0.93	1.41	0.59	3.88	3.03	2.26	1.31	0.92	1.31

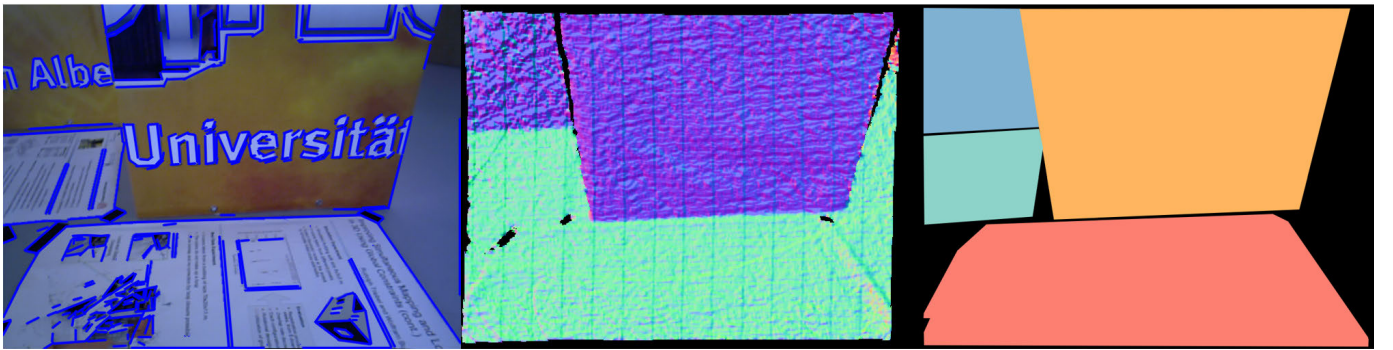


Fig. 3. Planar hypothesis construction. From left to right: RGB image with line features, normal map, and final planar segmentation.

is evident that our method produces more complete structures and sharper boundaries, while significantly reducing artifacts and blurring.

2) *Evaluation on TUM RGB-D*: We further evaluate the performance of our method on the TUM RGB-D dataset to verify its effectiveness in real-world environments. Since the raw depth data of this dataset contains significant noise, we do

not report Depth L1 results. As shown in Table II, our method achieves the best ATE across all sequences, demonstrating stable and accurate pose tracking in real scenarios. In terms of rendering quality, our approach also attains competitive results on PSNR, SSIM, and LPIPS. These findings confirm the robustness of the proposed method when applied to real-world data.

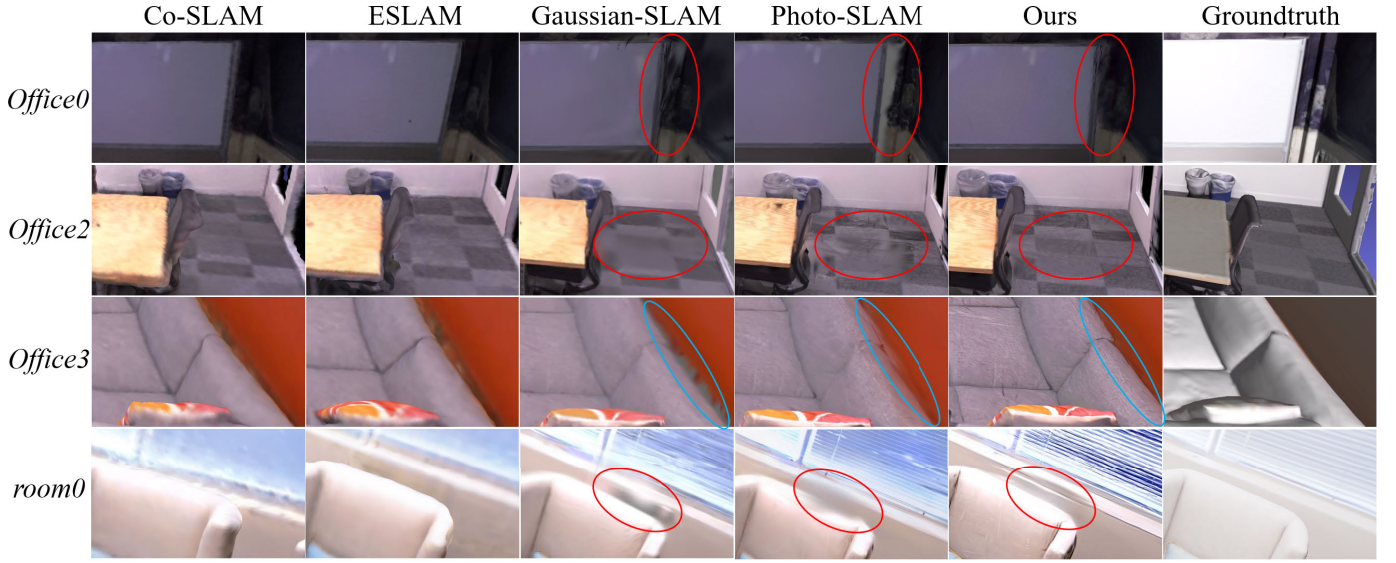


Fig. 4. Reconstruction results of partial sequences on the Replica dataset. It can be observed that our method yields more realistic results compared to NeRF-based methods. Furthermore, relative to 3DGS-based methods, our approach produces sharper edges and better preserves the geometric structure of the scene. This improvement mainly benefits from the incorporation of structural features during mapping: line and plane constraints provide strong geometric regularization in weak-texture regions, while the geometric densification strategy complements missing structures, leading to more complete and faithful reconstructions.

TABLE II

EVALUATION ON TUM RGB-D DATASET. BEST RESULTS ARE INDICATED AS **FIRST**, **SECOND**, AND **THIRD**. ATE IS REPORTED IN CM

Method	fr1-desk				fr2-xyz				fr3-office			
	ATE↓	PSNR↑	SSIM↑	LPIPS↓	ATE↓	PSNR↑	SSIM↑	LPIPS↓	ATE↓	PSNR↑	SSIM↑	LPIPS↓
ORB-SLAM3(RGB-D)	1.534	—	—	—	0.400	—	—	—	1.000	—	—	—
ESLAM(RGB-D)	3.359	17.50	0.561	0.484	31.448	22.23	0.727	0.233	25.808	19.11	0.616	0.359
Co-SLAM(RGB-D)	3.094	16.42	0.482	0.591	31.347	19.18	0.595	0.374	25.374	17.86	0.547	0.452
GO-SLAM(RGB-D)	2.119	15.79	0.531	0.538	31.788	16.12	0.534	0.419	26.802	16.50	0.566	0.569
NICE-SLAM(RGB-D)	19.317	12.00	0.417	0.510	36.103	18.20	0.603	0.313	25.309	16.34	0.548	0.386
Photo-SLAM(RGB-D)	1.539	20.97	0.743	0.228	0.984	21.07	0.726	0.166	1.257	19.59	0.692	0.239
SplaTAM(RGB-D)	3.350	22.00	0.860	0.230	1.240	24.50	0.950	0.100	5.160	21.90	0.880	0.200
MonoGS(RGB-D)	1.520	19.67	0.730	0.330	1.580	16.17	0.720	0.310	1.650	20.63	0.770	0.340
Ours(Mono+IMU)	1.437	22.58	0.770	0.207	0.707	24.63	0.844	0.116	1.153	22.31	0.859	0.198

TABLE III

EVALUATION ON SCANNET DATASET. BEST RESULTS ARE INDICATED AS **FIRST**, **SECOND**, AND **THIRD**

Method	0000		0059		0106		0169		0181		0207		Avg.	
	ATE↓	PSNR↑	ATE↓	PSNR↑	ATE↓	PSNR↑	ATE↓	PSNR↑	ATE↓	PSNR↑	ATE↓	PSNR↑	ATE↓	PSNR↑
ORB-SLAM3(RGB-D+IMU)	5.70	—	6.00	—	7.90	—	6.20	—	12.80	—	7.10	—	7.62	—
ESLAM(RGB-D)	7.54	15.70	8.52	14.48	7.39	15.44	8.17	14.56	9.13	14.22	5.61	17.32	7.73	15.29
GO-SLAM(RGB-D)	5.94	15.74	8.27	13.15	8.07	14.58	8.42	14.49	8.29	15.72	6.94	15.37	7.66	14.84
SplaTAM(RGB-D)	12.83	17.81	10.10	19.60	17.72	19.23	12.08	20.55	11.10	16.76	7.47	17.95	11.88	18.65
Photo-SLAM(RGB-D)	7.62	19.74	7.94	20.01	9.36	18.70	10.01	20.23	22.97	14.45	7.23	19.96	10.86	18.85
MonoGS(RGB-D)	15.94	19.76	13.03	19.25	19.44	20.18	10.44	20.57	12.23	20.25	10.46	20.62	13.59	20.11
Ours(Mono+IMU)	6.58	21.53	6.17	21.07	7.34	22.18	8.32	23.04	10.14	22.67	6.98	23.15	7.59	22.37

3) *Evaluation on ScanNet*: To evaluate the performance of our method in large-scale environments, we conducted experiments on the ScanNet dataset. As shown in Table III, our approach achieves the lowest ATE in most sequences as well as on the overall average, indicating that it can maintain stable and accurate camera tracking even in complex environments. In terms of rendering quality, our method also yields better overall PSNR results. Notably, in several challenging sequences, some existing methods suffer from tracking drift or

degraded rendering quality, whereas our approach consistently preserves high tracking accuracy and rendering fidelity, further validating its robustness in large-scale and diverse indoor environments. As illustrated in Fig. 5, our method excels at recovering both the texture details and structural integrity of geometric objects.

4) *Evaluation on OpenLORIS*: We further evaluate the tracking performance of our method on the office sequences of the OpenLORIS dataset. This dataset is collected by

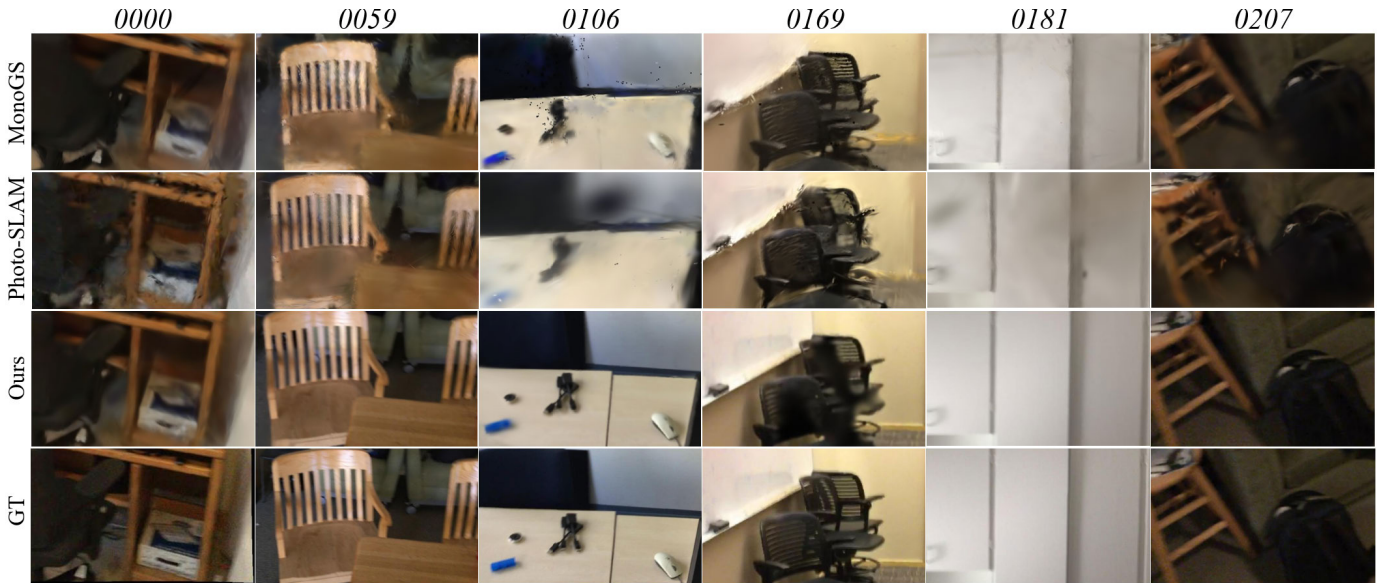


Fig. 5. Comparison of rendering results (zoom-in) on ScanNet sequences. Our method achieves fewer ghosting artifacts and sharper geometric boundaries.

TABLE IV

EVALUATION ON OPENLORIS OFFICE SEQUENCES IN TERMS OF ATE (CM). “X” INDICATES TRACKING FAILURE. BEST RESULTS ARE HIGHLIGHTED IN BOLD

Method	office1	office2	office3	office4	office5	office6	office7
ORB-SLAM3	6.31	8.92	11.24	15.37	21.26	6.21	18.33
Photo-SLAM	6.23	9.71	X	16.01	23.98	5.58	18.91
SplaTAM	6.54	9.12	X	X	23.50	5.16	16.24
MonoGS	6.72	10.03	31.45	15.56	X	5.28	X
Ours	5.99	8.55	10.96	15.34	20.87	5.09	16.12

mobile devices in real office environments and includes challenges such as low-texture regions, illumination changes, and dynamic disturbances, making camera localization extremely difficult. In such scenarios, many existing methods struggle to maintain stable pose estimation and even fail to track in certain sequences. As shown in Table IV, our method successfully completes tracking across all seven sequences (office1–7) and achieves lower ATE errors, outperforming other baseline methods overall. These results demonstrate that even in highly challenging real-world environments, our method can still deliver stable and reliable localization, highlighting its strong robustness.

5) *Evaluation on KITTI*: Although the proposed method is primarily designed for structured indoor environments, we further evaluate its performance on several outdoor sequences from the KITTI Odometry dataset to assess its applicability beyond indoor scenarios. Quantitative results on selected KITTI sequences are reported in Table V, and a qualitative reconstruction and rendering result on Sequence 07 is shown in Fig. 6.

As shown in Table V, our method achieves lower translational and rotational errors compared to baselines, while also providing improved rendering quality on most sequences. These results indicate that, despite being tailored for indoor

TABLE V

EVALUATION ON KITTI ODOMETRY SEQUENCES (t_{rel} IN % AND r_{rel} IN °/100m). BEST RESULTS ARE HIGHLIGHTED IN BOLD

Methods	Metrics	02	07	08
MonoGS	$t_{rel} \downarrow$	4.17	4.93	3.89
	$r_{rel} \downarrow$	1.62	1.12	1.07
	PSNR $\uparrow$	10.60	12.57	14.98
	SSIM $\uparrow$	0.33	0.44	0.52
	LPIPS $\downarrow$	0.84	0.78	0.74
Photo-SLAM	$t_{rel} \downarrow$	3.54	4.46	3.41
	$r_{rel} \downarrow$	1.44	0.99	0.90
	PSNR $\uparrow$	15.22	14.97	14.26
	SSIM $\uparrow$	0.44	0.51	0.48
	LPIPS $\downarrow$	0.66	0.55	0.66
Ours	$t_{rel} \downarrow$	2.62	1.43	2.16
	$r_{rel} \downarrow$	0.45	0.85	0.38
	PSNR $\uparrow$	20.01	19.86	20.29
	SSIM $\uparrow$	0.67	0.71	0.78
	LPIPS $\downarrow$	0.29	0.31	0.33

structured scenes, the proposed system remains effective in general outdoor environments.

We attribute this behavior to two main factors. First, outdoor urban scenes in the KITTI Odometry dataset often contain large man-made structures such as building facades, which provide sufficient structural regularities that can be captured by the proposed geometry-aware modeling. Second, vehicles typically move on paved roads, where the road surface and traffic markings introduce additional geometric and planar constraints that benefit both visual-inertial tracking and geometry-based mapping. Together, these properties allow the proposed method to generalize reasonably well to outdoor driving scenarios, even though such environments are not the primary target of this work.

C. Ablation Study

To analyze the contributions of different system components, we conduct ablation studies on selected sequences from

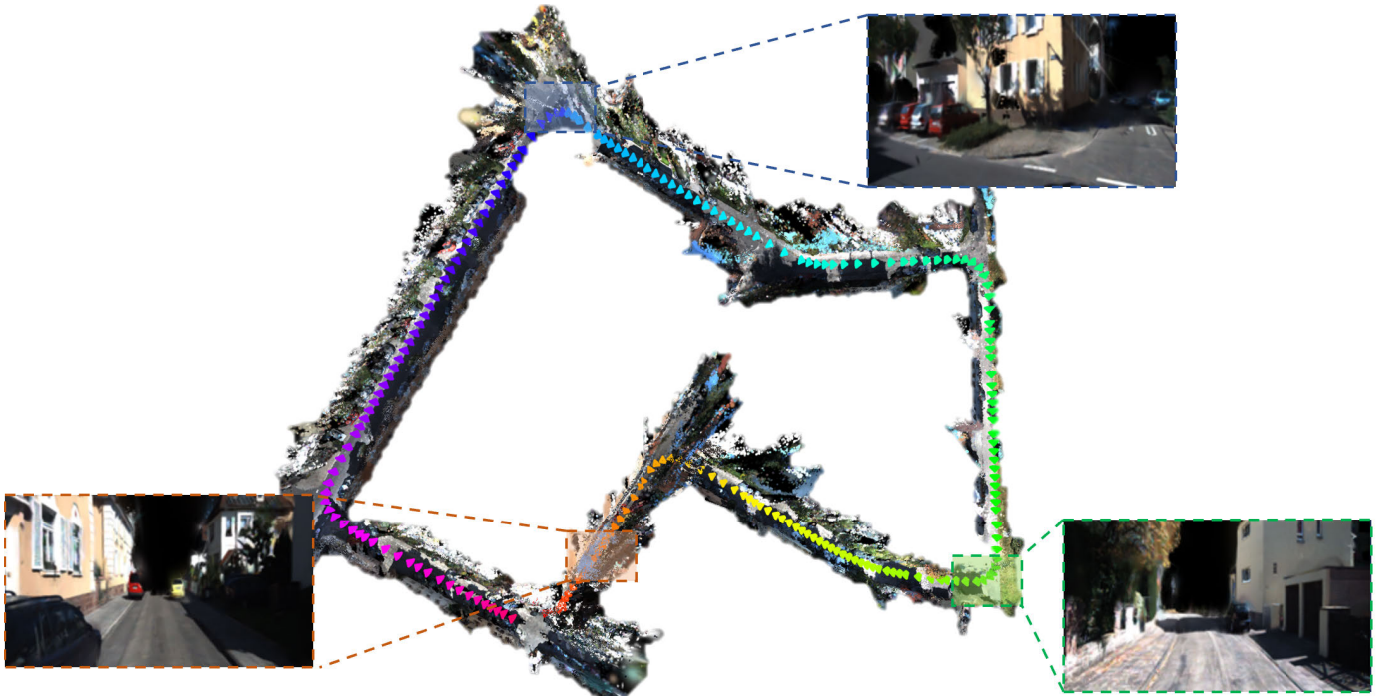


Fig. 6. Reconstruction results of the proposed method on Sequence 07 of the KITTI Odometry dataset. Despite being designed primarily for structured indoor environments, the proposed system is able to produce good reconstructions in outdoor driving scenarios, benefiting from the presence of man-made structures and road surface regularities.

TABLE VI

ABLATION ON EFFICIENT INITIALIZATION AND LINE FEATURES (MEAN OVER FIVE SEEDS). BASELINE: NO IMU AND LINE CONSTRAINTS; I: INTRODUCE IMU CONSTRAINTS FOR TRACKING ONLY; L: INTRODUCE LINE FEATURES CONSTRAINTS; E: USING OUR PROPOSED EFFICIENT INITIALIZATION. OURS: FULL SYSTEM WITH IMU INTEGRATION (TRACKING AND MAPPING), LINE FEATURES, AND STRUCTURED MAPPING. BEST RESULTS ARE HIGHLIGHTED IN BOLD

Variant	0000		0059		0106	
	ATE↓	PSNR↑	ATE↓	PSNR↑	ATE↓	PSNR↑
Baseline	7.62	19.74	7.94	20.01	9.36	18.70
Baseline+I	7.22	20.53	6.97	20.23	8.31	19.58
Baseline+I+E	7.03	20.55	7.06	19.97	8.37	19.81
Baseline+L	6.97	20.90	6.34	20.76	7.93	21.12
Ours	6.58	21.53	6.17	21.07	7.34	22.18

the ScanNet dataset. The baseline corresponds to a vision-only system without IMU or line constraints. As shown in Table VI, introducing IMU constraints for tracking alone improves both tracking accuracy and rendering quality compared to the baseline. By further enabling the proposed IMU initialization module, the system achieves more fast initialization, which helps improve convergence behavior during the early stages of mapping. In addition, incorporating line feature constraints also leads to performance gains, particularly in structured scenes, where line features provide complementary geometric cues beyond point-based constraints. The complete system consistently achieves the best performance across all sequences. These results demonstrate that IMU integration, line features, and structured mapping each contribute to the overall performance, and that their joint utilization is critical for achieving high-quality reconstruction.

TABLE VII

ABLATION ON DEPTH PREDICTION AND GEOMETRY PIPELINE ON REPLICA COMPLEX SCENES (APART0/1/2, FRL0/4). EACH SEQUENCE IS RUN FIVE TIMES, AND WE REPORT THE AVERAGE RESULT ACROSS ALL RUNS AND SEQUENCES. ASELINE: RGB + DEPTH PREDICTION (w/o IMU CONSTRAINED); D: IMU-CONSTRAINED DEPTH PREDICTION; G: GEOMETRY PIPELINE (PLANE HYPOTHESIS + PRUNING + DENSIFICATION). THE FULL METHOD CORRESPONDS TO BASELINE+D+G. BEST RESULTS ARE HIGHLIGHTED IN BOLD

Variant	PSNR↑	SSIM↑	LPIPS↓	Depth L1↓
Baseline	28.78	0.888	0.224	3.14
Baseline+D	29.37	0.900	0.211	2.76
Baseline+D+G	32.65	0.920	0.187	0.88

We further analyze the roles of IMU-constrained depth prediction and the geometric optimization module in mapping, and conduct ablation studies on challenging sequences from the Replica dataset. As shown in Table VII, compared with the baseline that relies solely on standard depth prediction, introducing IMU constraints effectively alleviates scale ambiguity and improves mapping quality. When geometric optimization is additionally incorporated, the system achieves significant gains in rendering quality. These results demonstrate that the geometric densification strategy enables the system to achieve more accurate and photorealistic reconstructions in weak-texture and structured scenes.

We evaluate the effects of different pyramid levels and geometric losses on the TUM RGB-D dataset, with results summarized in Table VIII. Here, memory denotes the storage size of the final Gaussian map, and FPS represents the system's average runtime speed. The results show that a three-level pyramid achieves faster optimization and runtime

TABLE VIII

ABLATION ON PYRAMID LEVELS AND GEOMETRIC LOSS ON TUM RGB-D DATASET. P_n : PYRAMID-BASED OPTIMIZATION WITH n LEVELS; GL: GEOMETRIC LOSS (LINE + PLANE CONSTRAINTS). BEST RESULTS ARE HIGHLIGHTED IN BOLD

Variant	PSNR $\uparrow$	Memory $\downarrow$	FPS $\uparrow$
P_3 w/ GL	22.34	142MB	20.7
P_4 w/o GL	21.95	181MB	18.7
P_4 w/ GL	23.17	186MB	18.3
P_5 w/ GL	23.41	213MB	15.6

TABLE IX

RUNTIME BREAKDOWN OF THE PROPOSED SYSTEM ON THE EUROC DATASET. (UNDER P_3)

Frontend (per frame)		Backend (per keyframe)	
Module	Time (ms)	Module	Time (ms)
Points extraction & tracking	11.87	Depth estimation	12.89
Lines extraction & tracking	15.32	Plane construction	17.27
Local VIO	25.76	Gaussian mapping	52.89

but suffers from certain deficiencies in rendering quality. In contrast, a five-level pyramid (P_5 w/ GL) improves rendering fidelity but incurs higher computational and storage costs. The four-level configuration offers a more balanced trade-off across all metrics, and when combined with geometric losses, it further enhances rendering quality while maintaining reasonable computational overhead. Overall, the four-level pyramid with geometric loss achieves the best balance between reconstruction quality and system efficiency.

To further clarify the real-time characteristics of the proposed system, we provide a detailed runtime breakdown of individual modules measured on the EuRoC dataset [57], as summarized in Table IX. In the frontend, point and line feature extraction and tracking are executed in parallel for each incoming frame, together with local visual-inertial tracking. In contrast, the backend modules, including depth estimation, planar hypothesis construction, and Gaussian-based mapping optimization, are only triggered at keyframes, significantly reducing their impact on the overall runtime. This design allows the system to maintain a high-frequency frontend for stable real-time tracking, while performing more computationally intensive geometry-aware mapping asynchronously in the backend. These results demonstrate that, although the complete Gaussian optimization pipeline is computationally demanding, the proposed frontend-backend decoupled architecture enables practical real-time operation by carefully balancing per-frame and per-keyframe computations.

We evaluate the sensitivity of the geometric constraint weights on the Replica dataset. As shown in Table X, the default configuration ($\lambda_p^{(N)}, \lambda_l^{(N)} = (0.15, 0.10)$) achieves the best overall performance, yielding the highest PSNR and SSIM as well as the lowest LPIPS. Increasing either the plane or line constraint beyond this range consistently degrades rendering quality, indicating that overly strong geometric regularization restricts photorealistic appearance modeling. Similarly, reducing both constraints leads to inferior performance due to insufficient structural guidance.

We investigate the robustness of the proposed method under different IMU quality conditions. In particular, we conduct

TABLE X

SENSITIVITY ANALYSIS OF GEOMETRIC CONSTRAINT WEIGHTS ON THE REPLICA DATASET. RESULTS ARE AVERAGED OVER FIVE RUNS ON EVERY SEQUENCES

$\lambda_p^{(N)}$	$\lambda_l^{(N)}$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
0.30	0.10	31.46	0.916	0.132
0.15	0.20	31.58	0.915	0.135
0.15	0.10	33.64	0.937	0.114
0.10	0.15	32.93	0.928	0.121
0.10	0.10	31.83	0.921	0.127

TABLE XI

IMU QUALITY SENSITIVITY ANALYSIS ON KITTI ODOMETRY SEQUENCE 07. WE COMPARE FIVE VARIANTS: **REAL**, WHICH USES THE REAL IMU MEASUREMENTS; **NO-IMU-SUPERVISED**, WHICH DISABLES IMU-SUPERVISED DEPTH; **SYN-L** ($0.5\times$), **SYN-N** ($1\times$), AND **SYN-H** ($2\times$), WHICH USE SYNTHETIC IMU MEASUREMENTS GENERATED FROM GROUND-TRUTH TRAJECTORIES WITH LOW, NOMINAL, AND HIGH NOISE LEVELS COMPARED WITH THE REAL IMU. (t_{REL} IN % AND r_{REL} IN $^\circ/100$ m)

Variant	$t_{rel} \downarrow$	$r_{rel} \downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Real	1.43	0.85	19.86	0.71	0.31
No-IMU-Supervised	1.43	0.85	15.25	0.53	0.56
Syn-L ($0.5\times$)	1.38	0.80	21.02	0.79	0.29
Syn-N ($1\times$)	1.40	0.82	20.43	0.74	0.30
Syn-H ($2\times$)	1.46	0.87	19.29	0.70	0.33

a controlled sensitivity study on Sequence 07 of the KITTI Odometry dataset, where we evaluate five variants: using the real IMU measurements, disabling IMU-supervised depth (No-IMU-Supervised), and using synthetic IMU measurements generated from ground-truth trajectories with three noise levels, i.e., Syn-L ($0.5\times$), Syn-N ($1\times$), and Syn-H ($2\times$), where the noise magnitudes are scaled relative to the real IMU. As shown in Table XI, the IMU-supervised depth estimation and associated geometry-driven mapping components substantially improve the reconstruction quality while the trajectory accuracy remains unchanged. Moreover, as the synthetic IMU noise increases from Syn-L to Syn-H, both odometry accuracy and reconstruction quality exhibit a mild degradation. The relatively limited performance drop indicates that the proposed method is robust to variations in IMU noise. Notably, the variant using the real IMU achieves performance comparable to the same synthetic setting (Syn-N), demonstrating that the proposed method remains effective under real sensor noise conditions. These results further validate the robustness of the proposed system to IMU noise variations and support its applicability in real-world scenarios beyond idealized IMU assumptions.

V. CONCLUSION

In this article, we have proposed a visual-inertial SLAM system for monocular cameras and IMUs that integrates multimodal information across both tracking and mapping stages. At the front end, an efficient initialization strategy jointly leverages point, line, and IMU observations to rapidly establish the global scale and orientation. Robust pose estimation is then achieved through factor-graph optimization that tightly

couples these constraints. At the back end, we extend the use of IMU and line features beyond tracking into mapping: IMU-supervised depth estimation and normal maps are fused with structural lines to generate planar hypotheses, which subsequently guide the geometry-driven densification of Gaussian primitives. In addition, we introduce a pyramid-based Gaussian optimization framework with multidimensional losses that progressively refine global geometric structures and local texture details.

Extensive experiments demonstrate that the proposed framework achieves state-of-the-art tracking and mapping results compared to existing methods. In particular, it shows clear advantages in weak-texture and structured environments, where conventional approaches often suffer from drift or leave holes in the reconstructed scene. A key factor behind these improvements lies in the comprehensive use of IMU measurements: beyond providing rapid and scale-consistent initialization, they also supervise depth prediction and contribute to geometry-constrained mapping, effectively reducing scale drift and enhancing structural fidelity. Although the system achieves promising results, there remain several limitations. First, it is less suited to highly dynamic environments, where moving objects may degrade both tracking and reconstruction quality. Second, loop closure detection and long-term consistency have not yet been fully integrated into the Gaussian optimization stage. We believe these issues can be addressed in future work.

REFERENCES

- [1] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-scale direct monocular SLAM," in *Proc. Eur. Conf. Comput. Vis.*, Zurich, Switzerland, Sep. 2014, pp. 834–849.
- [2] R. Wang, M. Schworer, and D. Cremers, "Stereo DSO: Large-scale direct sparse visual odometry with stereo cameras," in *Proc. IEEE Int. Conf. Comput. Vis.*, Sep. 2017, pp. 3903–3911.
- [3] C. Forster, M. Pizzoli, and D. Scaramuzza, "SVO: Fast semi-direct monocular visual odometry," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2014, pp. 15–22.
- [4] S. Leutenegger, P. Furgale, V. Rabaud, M. Chli, K. Konolige, and R. Siegwart, "Keyframe-based visual-inertial SLAM using nonlinear optimization," in *Proc. Robot., Sci. Syst. (RSS)*, Jun. 2013, pp. 1–8.
- [5] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multi-map SLAM," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.
- [6] B. Kerbl, G. Kopanas, T. Leimkuehler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 1–139, Aug. 2023.
- [7] H. Huang, L. Li, H. Cheng, and S.-K. Yeung, "Photo-SLAM: Real-time simultaneous localization and photorealistic mapping for monocular, stereo, and RGB-D cameras," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 21584–21593.
- [8] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, "Gaussian splatting SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Aug. 2024, pp. 18039–18048.
- [9] N. Keetha et al., "SplatTAM: Splat, track & map 3D Gaussians for dense RGB-D SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 21357–21366.
- [10] S. Liu et al., "MG-SLAM: Structure Gaussian splatting SLAM with Manhattan world hypothesis," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 17034–17049, 2025.
- [11] Z. Yang and S. Shen, "Monocular visual-inertial state estimation with online initialization and camera-IMU extrinsic calibration," *IEEE Trans. Autom. Sci. Eng.*, vol. 14, no. 1, pp. 39–51, Jan. 2017.
- [12] J. Jung, J. Han, H. An, J. Kang, S. Park, and S. Kim, "Relaxing accurate initialization constraint for 3D Gaussian splatting," 2024, *arXiv:2403.09413*.
- [13] E. Sandström et al., "Splat-SLAM: Globally optimized RGB-only SLAM with 3D Gaussians," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2025, pp. 1680–1691.
- [14] W. Yin et al., "Metric3D: Towards zero-shot metric 3D prediction from a single image," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 9009–9019.
- [15] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth anything: Unleashing the power of large-scale unlabeled data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 10371–10381.
- [16] Z. Li, S. F. Bhat, and P. Wonka, "PatchFusion: An end-to-end tile-based framework for high-resolution monocular metric depth estimation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 10016–10025.
- [17] S. F. Bhat, R. Birkel, D. Wofk, P. Wonka, and M. Müller, "ZoeDepth: Zero-shot transfer by combining relative and metric depth," 2023, *arXiv:2302.12288*.
- [18] Z. Xie, Z. Geng, J. Hu, Z. Zhang, H. Hu, and Y. Cao, "Revealing the dark secrets of masked image modeling," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2023, pp. 14475–14485.
- [19] S. Zhang, J. Zhang, and D. Tao, "Towards scale-aware, robust, and generalizable unsupervised monocular depth estimation by integrating IMU motion dynamics," in *Proc. Eur. Conf. Comput. Vis. Cham, Switzerland: Springer*, Oct. 2022, pp. 143–160.
- [20] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A versatile and accurate monocular SLAM system," *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.
- [21] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras," *IEEE Trans. Robot.*, vol. 33, no. 5, pp. 1255–1262, Oct. 2017.
- [22] T. Qin, P. Li, and S. Shen, "VINS-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.
- [23] P. Geneva, K. Eickenhoff, W. Lee, Y. Yang, and G. Huang, "OpenVINS: A research platform for visual-inertial estimation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Paris, France, May 2020, pp. 4666–4672.
- [24] Y. He, J. Zhao, Y. Guo, W. He, and K. Yuan, "PL-VIO: Tightly-coupled monocular visual-inertial odometry using point and line features," *Sensors*, vol. 18, no. 4, p. 1159, Apr. 2018.
- [25] Q. Fu et al., "PL-VINS: Real-time monocular visual-inertial SLAM with point and line features," 2020, *arXiv:2009.07462*.
- [26] Y. Li, N. Brasch, Y. Wang, N. Navab, and F. Tombari, "Structure-SLAM: Low-drift monocular SLAM in indoor environments," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 6583–6590, Oct. 2020.
- [27] L. Xu, H. Yin, T. Shi, D. Jiang, and B. Huang, "EPLF-VINS: Real-time monocular visual-inertial SLAM with efficient point-line flow features," *IEEE Robot. Autom. Lett.*, vol. 8, no. 2, pp. 752–759, Feb. 2023.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," *Commun. ACM*, vol. 65, no. 1, pp. 99–106, Jan. 2022.
- [29] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, "IMAP: Implicit mapping and positioning in real-time," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 6229–6238.
- [30] Z. Zhu et al., "NICE-SLAM: Neural implicit scalable encoding for SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 12776–12786.
- [31] X. Yang, H. Li, H. Zhai, L. Chen, W. Li, and F. Lu, "Vox-fusion: Dense tracking and mapping with voxel-based neural implicit representation," in *Proc. IEEE Int. Symp. Mixed Augmented Reality (ISMAR)*, Oct. 2022, pp. 499–507.
- [32] Y. Zhang, F. Tosi, S. Mattoccia, and M. Poggi, "GO-SLAM: Global optimization for consistent 3D instant reconstruction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 3727–3737.
- [33] H. Wang, J. Wang, and L. Agapito, "Co-SLAM: Joint coordinate and sparse parametric encodings for neural real-time SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 13293–13302.
- [34] M. M. Johari, C. Carta, and F. Fleuret, "ESLAM: Efficient dense SLAM system based on hybrid representation of signed distance fields," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 17408–17419.
- [35] T. Deng et al., "MCN-SLAM: Multi-agent collaborative neural SLAM with hybrid implicit neural scene representation," 2025, *arXiv:2506.18678*.
- [36] T. Deng et al., "MNE-SLAM: Multi-agent neural SLAM for mobile robots," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 1485–1494.

- [37] T. Deng et al., "PLGSLAM: Progressive neural scene representation with local to global bundle adjustment," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 19657–19666.
- [38] T. Deng et al., "NeSLAM: Neural implicit mapping and self-supervised feature tracking with depth completion and denoising," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 12309–12321, 2025.
- [39] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald, "Gaussian-SLAM: Photo-realistic dense SLAM with Gaussian splatting," 2023, *arXiv:2312.10070*.
- [40] M. Li et al., "SGS-SLAM: Semantic Gaussian splatting for neural dense SLAM," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 163–179.
- [41] S. Liu, X. Chen, H. Chen, Q. Xu, and M. Li, "DeRainGS: Gaussian splatting for enhanced scene reconstruction in rainy environments," in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, 2025, pp. 5558–5566.
- [42] M. Li, W. Chen, N. Cheng, J. Xu, D. Li, and H. Wang, "GARAD-SLAM: 3D Gaussian splatting for real-time anti dynamic SLAM," 2025, *arXiv:2502.03228*.
- [43] M. Li, S. Liu, T. Deng, and H. Wang, "DenseSplat: Densifying Gaussian splatting SLAM with neural radiance prior," 2025, *arXiv:2502.09111*.
- [44] H. Wang, J. Huang, L. Yang, T. Deng, G. Zhang, and M. Li, "LLGS: Unsupervised Gaussian splatting for image enhancement and reconstruction in pure dark environment," 2025, *arXiv:2503.18640*.
- [45] M. Li et al., "Dy3DGS-SLAM: Monocular 3D Gaussian splatting SLAM for dynamic environments," 2025, *arXiv:2506.05965*.
- [46] C. Akinlar and C. Topal, "EDLines: A real-time line segment detector with a false detection control," *Pattern Recognit. Lett.*, vol. 32, no. 13, pp. 1633–1642, Oct. 2011.
- [47] L. Kneip and H. Li, "Efficient computation of relative pose for multi-camera systems," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2014, pp. 446–453.
- [48] D. Zuniga-Noel, F.-A. Moreno, and J. Gonzalez-Jimenez, "An analytical solution to the IMU initialization problem for visual-inertial systems," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 6116–6122, Jul. 2021.
- [49] Y. Zhou, L. Kneip, C. Rodríguez, and H. Li, "Divide and conquer: Efficient density-based tracking of 3D sensors in Manhattan worlds," in *Proc. Comput. Vis. (ACCV)*, Nov. 2017, pp. 3–19.
- [50] B. Huang, Z. Yu, A. Chen, A. Geiger, and S. Gao, "2D Gaussian splatting for geometrically accurate radiance fields," in *Proc. Special Interest Group Comput. Graph. Interact. Techn. Conf. Papers*, Jul. 2024, pp. 1–11.
- [51] J. Straub et al., "The replica dataset: A digital replica of indoor spaces," 2019, *arXiv:1906.05797*.
- [52] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2012, pp. 573–580.
- [53] A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nießner, "ScanNet: Richly-annotated 3D reconstructions of indoor scenes," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2432–2443.
- [54] X. Shi et al., "Are we ready for service robots? The OpenLORIS-scene datasets for lifelong SLAM," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2020, pp. 3139–3145.
- [55] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2012, pp. 3354–3361.
- [56] A. Patron-Perez, S. Lovegrove, and G. Sibley, "A spline-based trajectory representation for sensor fusion and rolling shutter cameras," *Int. J. Comput. Vis.*, vol. 113, no. 3, pp. 208–219, Jul. 2015.
- [57] M. Burri et al., "The EuRoC micro aerial vehicle datasets," *Int. J. Robot. Res.*, vol. 35, no. 10, pp. 1157–1163, Sep. 2016.



Ruijie Che received the B.S. degree in remote sensing science and technology from Wuhan University, Wuhan, China, in 2022. He is currently pursuing the Ph.D. degree in signal and information processing with the University of Chinese Academy of Sciences, Beijing, China.

His current research interests focus on multi-sensor integration for positioning, mapping, and perception.



Hao Hu received the M.S. degree from Shandong Jiaotong University, Jinan, China, in 2024. He is currently pursuing the Ph.D. degree in computer science and technology with Shandong Normal University, Jinan.

His current research interests focus on perception and obstacle-avoidance planning algorithms for autonomous driving.



Ningbo Wang is currently an Associate Professor at the Aerospace Information Research Institute, Chinese Academy of Sciences (CAS), Beijing, China. His research interests include multifrequency and multiconstellation GNSS bias correction, real-time precise positioning, and multisensor integrated navigation.



Zishen Li received the Ph.D. degree in geodesy and surveying engineering from the Institute of Geodesy and Geophysics, Chinese Academy of Sciences (CAS), Beijing, China, in 2013.

He is currently a Professor at the Aerospace Information Research Institute, CAS. His research interests include real-time ionospheric remote sensing, high-precision GNSS positioning, and multisensor integrated navigation.